

Enterprise AI Knowledge Assistant

Generative AI and ML Capstone Project Submission

Full-stack enterprise knowledge assistant using RAG, LangGraph, OpenAI, ChromaDB, evaluation, observability, and source attribution.

Table of Contents

<i>What the application does</i>	<i>4</i>
1. Document ingestion	4
2. Question answering	4
<i>High-level architecture</i>	<i>5</i>
<i>Technology stack.....</i>	<i>6</i>
<i>Repository structure</i>	<i>6</i>
<i>Document ingestion flow</i>	<i>7</i>
Step 1: Upload	7
Step 2: Validate.....	7
Step 3: Store	7
Step 4: Detect duplicates.....	7
Step 5: Select a loader	7
Step 6: Load and enrich content.....	8
Step 7: Chunk the document	8
Step 8: Create embeddings and index.....	8
<i>Agentic question-answering flow</i>	<i>8</i>
Planner	9
Retrieval.....	9
No-context protection	9
Reasoning	10
Validation and source attribution.....	10
Final LangGraph flow	10
<i>Conversation memory.....</i>	<i>11</i>
<i>Streaming responses.....</i>	<i>11</i>
<i>Document management.....</i>	<i>11</i>
List documents.....	11
Delete a document.....	11
<i>API summary</i>	<i>12</i>
<i>Observability</i>	<i>12</i>
<i>AI evaluation.....</i>	<i>13</i>
Answer correctness	13

Source match.....	13
Unanswerable behavior	13
LangSmith Evaluation Analysis	14
1. Evaluation Results Analysis	14
2. Key Metrics from the Latest Evaluation	14
3. Improvement Across Successful Runs	14
4. Evaluation Summary.....	15
Configuration	15
Running the backend locally	16
Requirements.....	16
Running the frontend locally.....	17
Requirements.....	17
Running tests.....	17
Running the LangSmith evaluation	18
Design choices.....	18
Why LangGraph?	18
Why ChromaDB?	19
Why a retrieval threshold?	19
Why source attribution?	19
Why duplicate detection?	19
Current implementation notes	19
Demo	19
Future Enhancements.....	36
AWS Deployment	36
Persistent Application Storage.....	36
Authentication and Role-Based Access	37
AWS Secrets Manager.....	37
Production Target Architecture.....	38
AWS Architecture diagram	39

A full-stack enterprise knowledge assistant that lets users upload internal documents and ask questions about their content.

The application combines Retrieval-Augmented Generation, LangGraph-based agentic orchestration, OpenAI models, ChromaDB vector search, conversation memory, source attribution, streaming responses, evaluation, and observability.

The main goal is simple: give users useful answers from their own documents while avoiding unsupported answers when relevant information cannot be found.

What the application does

The application supports two main workflows.

1. Document ingestion

Users can upload supported business documents. The backend validates the file, stores it locally, checks for duplicate content, reads the document, breaks the content into smaller chunks, and indexes those chunks in ChromaDB.

Supported file formats:

- PDF
- TXT
- CSV
- XLSX

Once a document is indexed, its content can be searched when users ask questions.

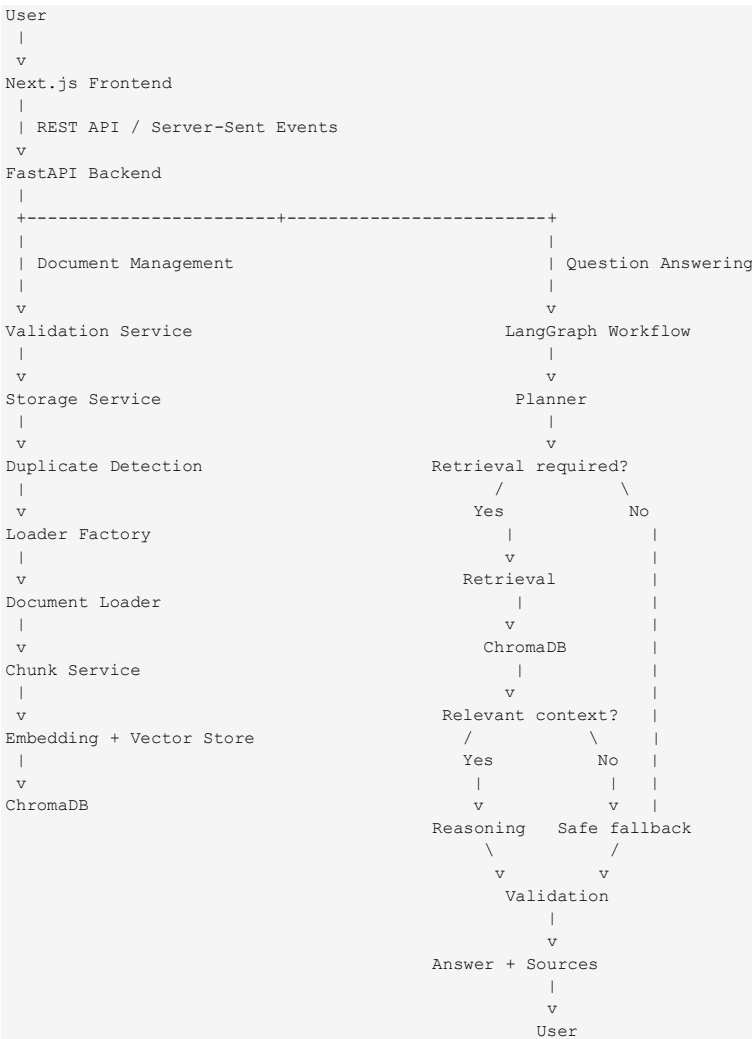
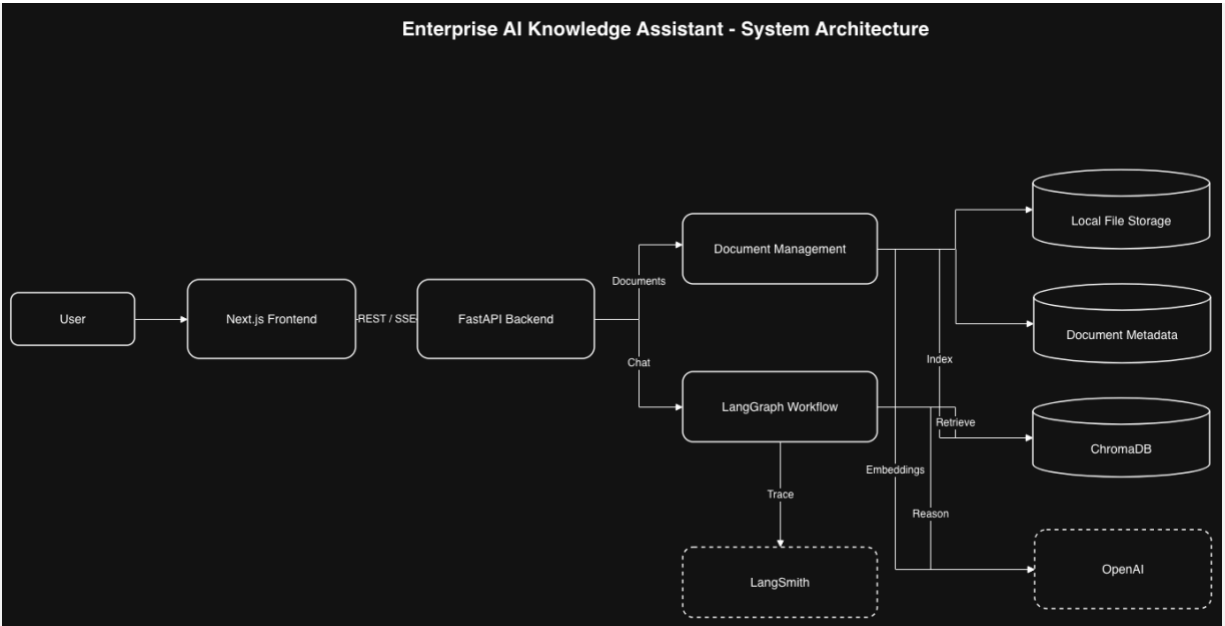
2. Question answering

Users ask questions through the application. The backend runs the question through a LangGraph workflow.

The workflow decides whether document retrieval is needed, searches ChromaDB for relevant document chunks, builds a grounded prompt, asks the OpenAI language model to generate an answer, validates the result, and returns source information with the response.

If the system does not find sufficiently relevant document content, it does not send unrelated context to the language model. Instead, it returns a safe response explaining that the information could not be found in the provided documents.

High-level architecture



Technology stack

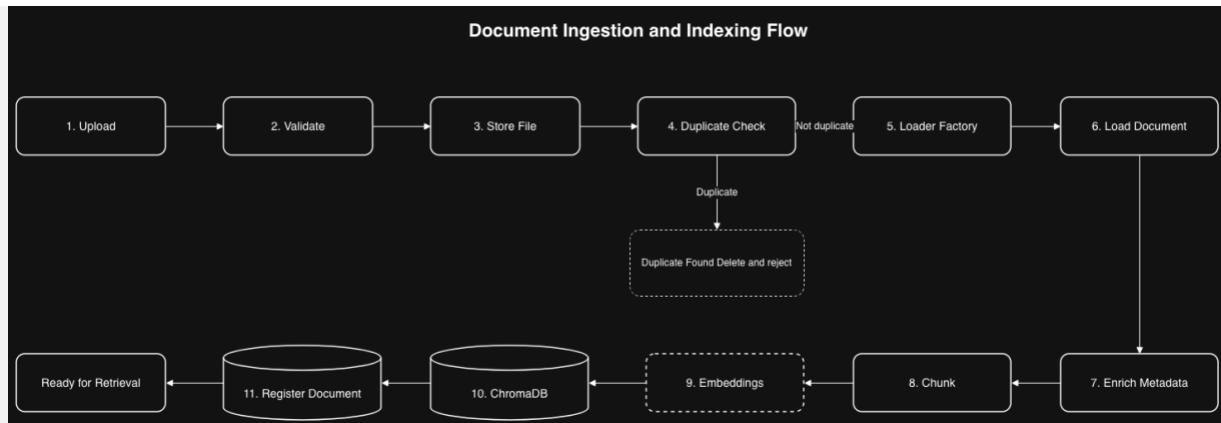
Area	Technology
Frontend	Next.js 16, React 19, TypeScript
UI	Tailwind CSS 4, shadcn, Base UI
Frontend animation	Framer Motion
Backend API	FastAPI
Agent orchestration	LangGraph
LLM integration	OpenAI through LangChain
Embeddings	OpenAI text-embedding-3-small
Vector database	ChromaDB
RAG framework	LangChain components with custom services
Conversation state	In-memory conversation memory
Evaluation	LangSmith datasets and evaluators
Observability	Application logging and LangSmith tracing
Document metadata	Local document repository
File storage	Local filesystem
Testing	pytest

Repository structure

```
enterprise-ai-knowledge-assistant/  
|  
+-- backend/  
|   +-- app/  
|   |   +-- ai/  
|   |   |   +-- agentic/  
|   |   |   +-- chunking/  
|   |   |   +-- embeddings/  
|   |   |   +-- evaluation/  
|   |   |   +-- indexing/  
|   |   |   +-- llm/  
|   |   |   +-- loaders/  
|   |   |   +-- memory/  
|   |   |   +-- models/  
|   |   |   +-- retrieval/  
|   |   |   +-- vectorstores/  
|   |   +-- api/  
|   |   +-- core/  
|   |   +-- data/  
|   |   +-- repositories/  
|   |   +-- schemas/  
|   |   +-- scripts/  
|   |   +-- services/  
|   |   +-- dependencies.py  
|   |   +-- main.py  
|   +-- tests/  
|   +-- Makefile  
|   +-- requirements.txt  
|  
+-- frontend/  
|   +-- app/  
|   +-- components/  
|   +-- hooks/  
|   +-- public/  
|   +-- services/  
|   +-- types/  
|   +-- utils/  
|   +-- package.json  
|  
+-- docs/  
+-- LICENSE  
+-- README.md
```

Document ingestion flow

The ingestion pipeline prepares uploaded documents for later semantic search.



Step 1: Upload

A client sends a document to:

```
POST /api/v1/upload
```

The FastAPI route passes the uploaded file to `DocumentService`.

Step 2: Validate

`ValidationService` checks the uploaded file before it is accepted.

The current configuration allows PDF, TXT, CSV, and XLSX files, with a default maximum upload size of 25 MB.

Step 3: Store

`StorageService` saves the accepted file in the configured upload directory.

The storage layer assigns a document identifier and keeps the original filename as metadata.

Step 4: Detect duplicates

`DocumentService` calculates a SHA-256 hash of the stored file.

The hash is checked against the document repository. If the same content has already been uploaded, the newly stored copy is removed and the upload is rejected as a duplicate.

Step 5: Select a loader

`LoaderFactory` selects a loader based on the file extension.

```
.pdf -> PdfLoader
.txt -> TxtLoader
.csv -> CsvLoader
.xlsx -> ExcelLoader
```

Step 6: Load and enrich content

The selected loader converts the document into LangChain `Document` objects.

Storage metadata such as `document_id`, original filename, and stored filename is added before chunking.

Step 7: Chunk the document

`ChunkService` divides the loaded content into smaller pieces.

The default configuration uses:

```
Chunk size: 1000
Chunk overlap: 200
```

Chunking allows the retrieval system to find the most relevant parts of a large document instead of passing the entire file to the language model.

Step 8: Create embeddings and index

The document chunks are added to the vector store.

The project uses ChromaDB as the vector database and OpenAI embeddings. The default embedding model is:

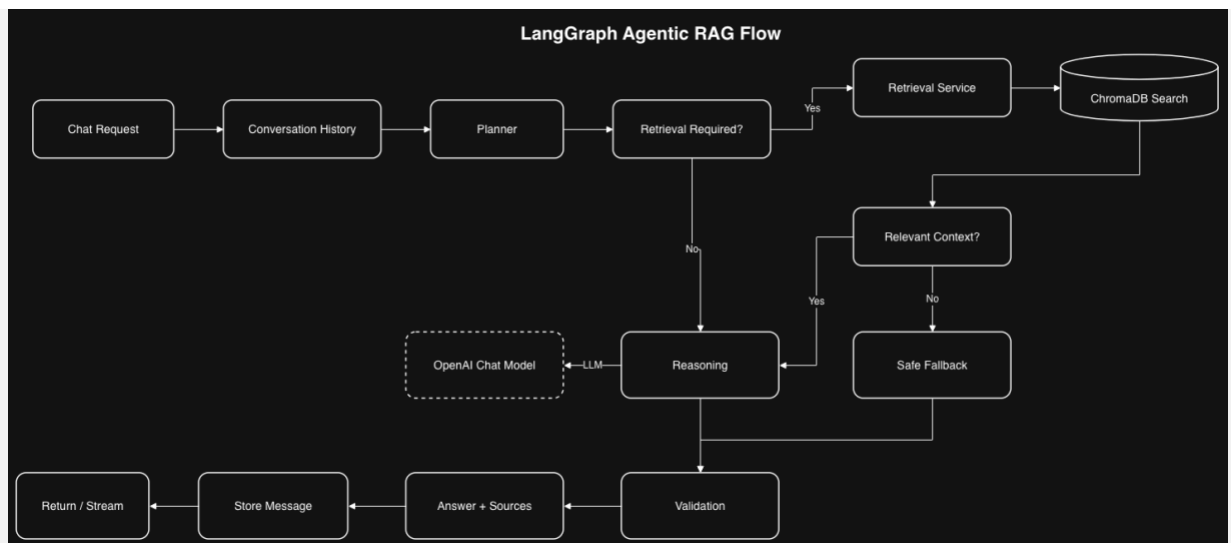
```
text-embedding-3-small
```

After indexing is complete, the document is registered with metadata including upload time, content type, size, chunk count, and indexing status.

Agentic question-answering flow

The question-answering workflow is implemented with LangGraph.

The graph uses shared `GraphState` so each node can add information to the same request state.



Important state includes:

```
question
request_id
intent
retrieval_strategy
top_k
requires_retrieval
retrieved_chunks
prompt
answer
validated
confidence_score
sources
metadata
conversation_history
```

Planner

The Planner is the first node.

Its job is to understand the request and decide how the workflow should proceed.

It can determine:

- request intent
- retrieval strategy
- number of results to retrieve
- whether document retrieval is required

After planning, LangGraph conditionally routes the request.

```
START
|
v
Planner
|
+---- retrieval required ----> Retrieval
|
+---- retrieval not required -> Reasoning
```

Retrieval

When retrieval is required, `RetrievalService` searches ChromaDB for semantically similar document chunks.

The current default settings include:

```
Default top_k:          5
Similarity score threshold: 0.15
```

Retrieved chunks below the configured threshold are removed.

This reduces the chance of passing unrelated document content to the language model.

No-context protection

After retrieval, the graph checks whether relevant content was found.

If relevant content exists:

```
Retrieval -> Reasoning
```

If no relevant content exists:

```
Retrieval -> No Context -> Validation
```

The no-context node skips LLM reasoning and returns:

```
I couldn't find that information in the provided documents.
```

This is an important guardrail because it reduces unsupported answers when the knowledge base does not contain useful evidence.

Reasoning

The Reasoning node uses the retrieved context, user question, and conversation history to build the final prompt.

`PromptBuilder` prepares the prompt and `LLMService` sends it to the configured OpenAI chat model.

The default configured chat model is:

```
gpt-5
```

The generated response is stored in the graph state.

Validation and source attribution

The Validation node runs after reasoning or after the no-context path.

It prepares source information from the retrieved documents and adds it to the response state.

This allows the API to return both the answer and the documents used to support that answer.

Final LangGraph flow

```
START
|
v
Planner
|
+-----+
| retrieval          | no retrieval
v                |
Retrieval          |
|                  |
v                  |
Relevant context?  |
|                  |
+-- Yes --> Reasoning <--+
|
+-- No --> No Context
|
v
Validation
|
v
END
```

Conversation memory

The workflow accepts a `session_id` with each chat request.

Conversation history is loaded before processing the current question and is passed through `GraphState`.

The user message is stored before workflow execution, and the assistant answer is stored after successful completion.

This allows follow-up questions to use previous conversation context within the same application session.

The current implementation uses application-level conversation memory rather than an external persistent conversation database.

Streaming responses

The backend supports both normal and streaming chat APIs.

Standard request:

```
POST /api/v1/chat
```

Streaming request:

```
POST /api/v1/chat/stream
```

The streaming endpoint uses Server-Sent Events.

During execution, the workflow can stream LLM tokens and LangGraph updates. The endpoint sends a final `done` event when processing completes.

This gives the frontend a better chat experience because users can see the response while it is being generated.

Document management

The project also exposes APIs for document management.

List documents

```
GET /api/v1/documents
```

Returns registered documents and metadata such as filename, content type, size, upload time, chunk count, and indexing status.

Delete a document

```
DELETE /api/v1/documents/{document_id}
```

Deletion removes:

1. ChromaDB chunks associated with the document
2. The physical uploaded file

3. The document metadata record

This prevents deleted documents from continuing to appear in future retrieval results.

API summary

Method	Endpoint	Purpose
POST	/api/v1/upload	Upload and index a document
GET	/api/v1/documents	List indexed documents
DELETE	/api/v1/documents/{document_id}	Delete a document and its indexed data
POST	/api/v1/chat	Ask a question and receive a complete response
POST	/api/v1/chat/stream	Ask a question and receive an SSE streaming response
GET	/api/v1/health	Application health check

FastAPI also provides interactive API documentation when the backend is running:

```
http://localhost:8000/docs
```

Observability

The backend records operational information such as:

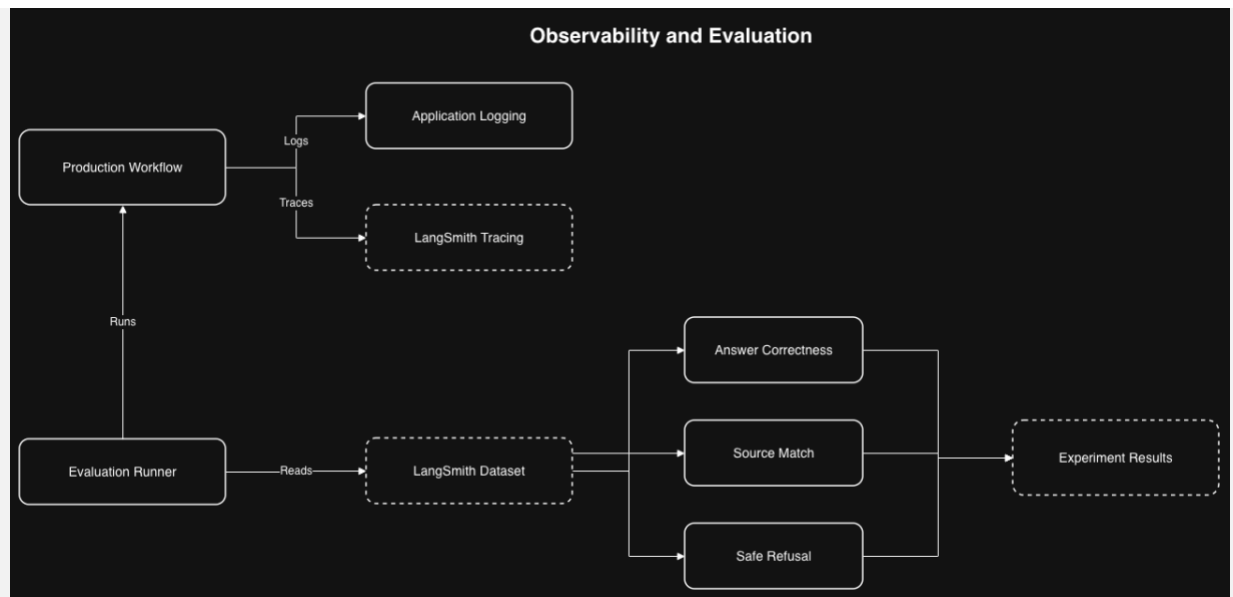
- request identifier
- session identifier
- workflow duration
- node execution information
- retrieval scores
- document source and page
- workflow success or failure
- indexing activity

Each workflow execution receives a unique `request_id`.

The project also supports LangSmith tracing through configuration variables.

```
LANGSMITH_TRACING  
LANGSMITH_API_KEY  
LANGSMITH_PROJECT
```

LangSmith makes it possible to inspect LLM and LangGraph execution outside normal application logs.



AI evaluation

The project contains an evaluation module under:

```
backend/app/ai/evaluation/
```

The evaluation runner executes test questions through the same production `AgenticWorkflow` used by the API.

The current evaluation checks three areas:

Answer correctness

An LLM judge compares the generated answer with a reference answer and returns a correctness score.

Source match

The evaluator checks whether the expected source appears in the returned source list.

Unanswerable behavior

Questions marked as unanswerable are checked to make sure the assistant safely indicates that enough information is not available.

Evaluation experiments are sent to LangSmith using the dataset:

```
enterprise-ai-rag-evaluation
```

This provides a repeatable way to measure changes to retrieval, prompts, models, and workflow logic.

LangSmith Evaluation Analysis

1. Evaluation Results Analysis

Run	Progress	Answer Correctness	Safe Refusal	Source Match	Error Rate	P50 Latency	P99 Latency	Total Tokens	Cost	Analysis
#1	0/15	N/A	N/A	N/A	0%	N/A	N/A	0	\$0.00	Initial experiment was created, but no evaluation cases were executed.
#2	15/15	0.00	1.00	0.20	100%	0.00s	0.00s	0	\$0.00	First attempted evaluation failed at execution level. Safe-refusal scoring remained high, but correctness and source attribution were ineffective because evaluated responses were not successfully produced.
#3	15/15	0.00	1.00	0.20	100%	2.45s	8.63s	9,073	\$0.05	The AI workflow began executing and consuming model tokens, but the evaluation still reported failures, indicating an integration or evaluator issue.
#4	15/15	0.73	1.00	0.87	0%	2.55s	6.52s	8,200	\$0.04	Major turning point. Execution errors were eliminated. Answer correctness reached 73% and source matching reached 87%, while safe-refusal behavior remained perfect.
#5	15/15	0.87	1.00	0.93	0%	3.30s	5.56s	13,620	\$0.05	Further improvements increased answer correctness to 87% and source matching to 93%. P99 latency also improved despite higher token usage.
#6	15/15	0.87	1.00	0.87	0%	3.17s	5.67s	13,623	\$0.05	Correctness remained stable at 87%, while source matching dropped slightly to 87%, suggesting some retrieval or source-attribution variation.
#7	15/15	0.93	1.00	0.93	0%	2.93s	9.35s	14,582	\$0.05	Best overall quality result. Answer correctness reached 93%, safe refusal stayed at 100%, and source matching reached 93%. The tradeoff was higher token usage and increased P99 tail latency.

2. Key Metrics from the Latest Evaluation

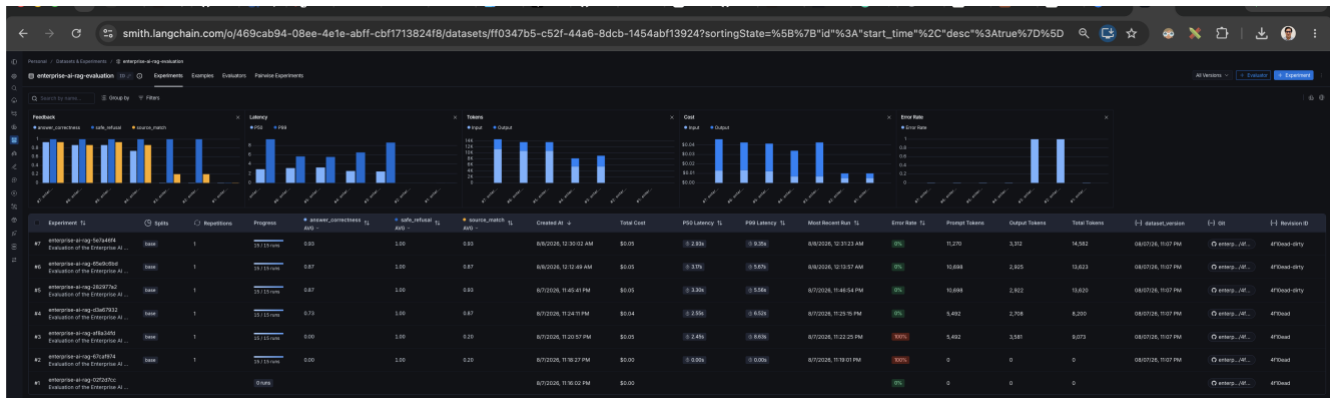
Metric	Latest Result	Interpretation
Answer Correctness	0.93 / 93%	Generated answers closely match the expected or reference answers for most evaluation cases.
Safe Refusal	1.00 / 100%	The system successfully handles questions that should not be answered from the available documents, validating the no-context guardrail.
Source Match	0.93 / 93%	The RAG pipeline generally identifies and attributes the expected supporting document sources.
Error Rate	0%	All 15 evaluation runs completed successfully.
P50 Latency	2.93 sec	Half of the evaluated requests completed within approximately three seconds.
P99 Latency	9.35 sec	A small number of slower requests significantly increase tail latency, providing an area for future optimization.
Prompt Tokens	11,270	Input and context processing represent the majority of token consumption.
Output Tokens	3,312	Generated responses account for a smaller portion of total token usage.
Total Tokens	14,582	Token consumption increased as the workflow and evaluation became more complete.
Total Cost	\$0.05	The 15-case evaluation remains inexpensive despite increased token usage.

3. Improvement Across Successful Runs

Metric	Run #4	Run #7	Change
Answer Correctness	73%	93%	+20 percentage points
Safe Refusal	100%	100%	Maintained
Source Match	87%	93%	+6 percentage points
Error Rate	0%	0%	Maintained
P50 Latency	2.55s	2.93s	+0.38s
P99 Latency	6.52s	9.35s	+2.83s
Total Tokens	8,200	14,582	+6,382
Cost	\$0.04	\$0.05	+\$0.01

4. Evaluation Summary

The LangSmith evaluation demonstrates progressive improvement in the Enterprise AI Knowledge Assistant. Early experiments exposed workflow and evaluation integration failures, including a 100% execution error rate. After resolving these issues and refining the RAG and agentic workflow, the system achieved a 0% error rate across all 15 evaluation cases. The latest evaluation achieved 93% answer correctness, 100% safe-refusal accuracy, and 93% source-match accuracy. These results indicate that the system can generate grounded answers, correctly associate answers with supporting document sources, and safely refuse questions when sufficient document context is unavailable. The evaluation also highlights an optimization opportunity, improvements in answer quality were accompanied by increased token consumption and higher P99 latency.



Configuration

Backend configuration is managed through environment variables using Pydantic Settings.

Create an environment file inside the `backend` directory.

```
cd backend
cp .env.example .env
```

A typical local configuration can include:

```
APP_NAME=Enterprise AI Knowledge System
APP_VERSION=0.1.0
ENVIRONMENT=development

HOST=0.0.0.0
PORT=8000

OPENAI_API_KEY=your_openai_api_key
CHAT_MODEL=gpt-5
EMBEDDING_MODEL=text-embedding-3-small

UPLOAD_DIR=app/uploads
CHROMA_DB_DIR=app/chroma_db

MAX_UPLOAD_SIZE_MB=25
LOG_LEVEL=INFO

LANGSMITH_TRACING=false
LANGSMITH_API_KEY=
LANGSMITH_PROJECT=enterprise-ai-knowledge-assistant
```

⊖ Do not commit real API keys.

Running the backend locally

Requirements

- Python 3.11 or compatible project version
- OpenAI API key

Create and activate a virtual environment.

```
cd backend  
  
python -m venv .venv  
source .venv/bin/activate
```

Install dependencies.

```
pip install -r requirements.txt
```

Create `.env` from the example file and add your OpenAI API key.

Run FastAPI:

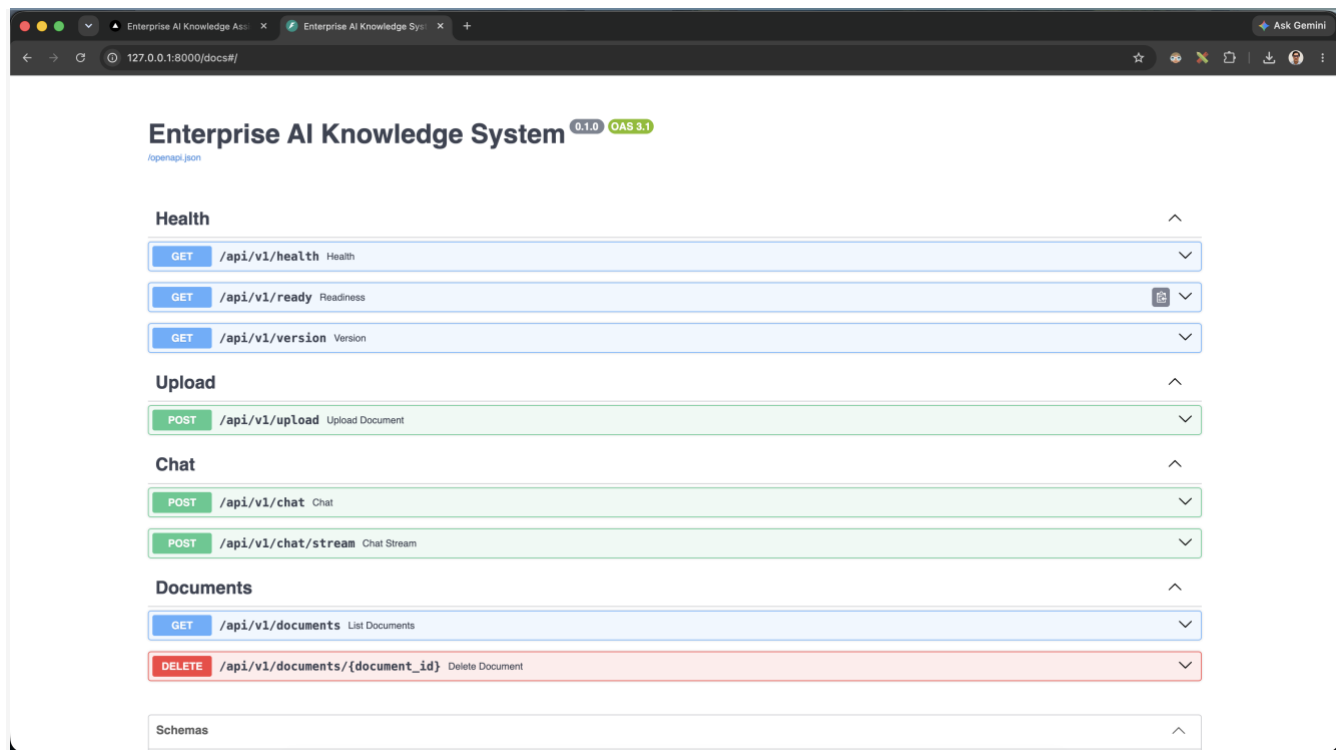
```
uvicorn app.main:app --reload --env-file .env
```

The API will be available at:

```
http://localhost:8000
```

Swagger documentation:

```
http://localhost:8000/docs
```

Running the frontend locally

Requirements

- Node.js
- npm

Install dependencies:

```
cd frontend
npm install
```

Start the development server:

```
npm run dev
```

Open:

```
http://localhost:3000
```

The FastAPI backend currently allows local frontend requests from `http://localhost:3000`.

Running tests

From the backend directory:

```
pytest
```

The repository contains tests for API behavior and AI workflow components.

Tests should be run after changes to retrieval, prompting, graph routing, validation, indexing, or service logic.

```
(backend) deepak@Deepaks-MacBook-Pro: ~/Documents/deepak/git/enterprise-ai-knowledge-assistant/backend
test session starts
platform darwin -- Python 3.11.14, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/deepak/Documents/deepak/git/enterprise-ai-knowledge-assistant/backend
configfile: pytest.ini
testpaths: tests
plugins: asyncio-1.4.0, langsmith-0.10.15, anyio-4.14.2
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 130 items

tests/ai/agent/test_agent_workflow_smoke.py . [ 0%]
tests/ai/agent/test_graph_state.py ..... [ 6%]
tests/ai/agent/test_planner_node.py ..... [ 10%]
tests/ai/agent/test_reasoning_node.py ..... [ 14%]
tests/ai/agent/test_retrieval_node.py ..... [ 18%]
tests/ai/agent/test_validation_node.py ..... [ 33%]
tests/ai/agent/test_workflow.py ..... [ 46%]
tests/ai/chunking/test_chunk_service.py .. [ 47%]
tests/ai/embeddings/test_embedding_service.py .. [ 49%]
tests/ai/llm/test_llm_service.py ..... [ 56%]
tests/ai/llm/test_prompt_builder.py ..... [ 63%]
tests/ai/memory/test_conversation_memory.py .... [ 66%]
tests/ai/retrieval/test_rag_service.py ... [ 68%]
tests/ai/retrieval/test_source_attribution.py .... [ 71%]
tests/ai/vectorstores/test_vector_store_service.py ..... [ 77%]
tests/api/test_chat.py ... [ 80%]
tests/api/test_documents.py ... [ 82%]
tests/api/test_health.py ... [ 84%]
tests/core/test_config.py . [ 85%]
tests/core/test_startup.py ... [ 87%]
tests/repositories/test_document_repository.py ..... [ 92%]
tests/schemas/test_chat_schema.py .... [ 95%]
tests/services/test_document_service.py ..... [100%]

===== warnings summary =====
./venv/lib/python3.11/site-packages/fastapi/testclient.py:1
/Users/deepak/Documents/deepak/git/enterprise-ai-knowledge-assistant/backend/.venv/lib/python3.11/site-packages/fastapi/testclient.py:1:
StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient # noqa

app/ai/loaders/pdf_loader.py:3
/Users/deepak/Documents/deepak/git/enterprise-ai-knowledge-assistant/backend/app/ai/loaders/pdf_loader.py:3: DeprecationWarning: `langchain-community` is being sunset and is no longer actively maintained. See https://github.com/langchain-ai/langchain-community/issues/674 for details and migration guidance toward standalone integration packages.
  from langchain_community.document_loaders import PyPDFLoader

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 130 passed, 2 warnings in 4.37s =====
(backend) deepak@Deepaks-MacBook-Pro: ~/Documents/deepak/git/enterprise-ai-knowledge-assistant/backend
```

Running the LangSmith evaluation

Make sure the required LangSmith and OpenAI environment variables are configured.

From the backend directory:

```
python -m app.ai.evaluation.runner
```

The runner executes the evaluation dataset against the production agentic workflow and sends the results to LangSmith.

Design choices

Why LangGraph?

The question-answering pipeline is more than a single prompt.

The application needs to plan, retrieve information, decide what to do when nothing useful is found, reason over context, validate the result, and preserve workflow metadata.

LangGraph makes those steps explicit and allows conditional routing between them.

Why ChromaDB?

ChromaDB provides local vector storage that works well for development and experimentation.

The application wraps vector operations behind `VectorStoreService`, which keeps the higher-level retrieval code less dependent on the specific vector database implementation.

Why a retrieval threshold?

A vector search will usually return something even when the returned chunks are weak matches.

The similarity threshold filters weak results before reasoning.

If nothing passes the threshold, the system uses its safe no-context flow rather than asking the model to generate an answer from unrelated content.

Why source attribution?

A useful enterprise assistant should make it possible to understand where an answer came from.

The validation stage attaches source metadata so consumers of the API can show supporting document information next to generated answers.

Why duplicate detection?

Uploading the same document multiple times would create duplicate vector chunks and could distort retrieval results.

The application calculates a SHA-256 content hash during upload and rejects content that already exists in the document registry.

Current implementation notes

This project is designed as a production-oriented learning and reference implementation, but some components are intentionally local.

Current local components include:

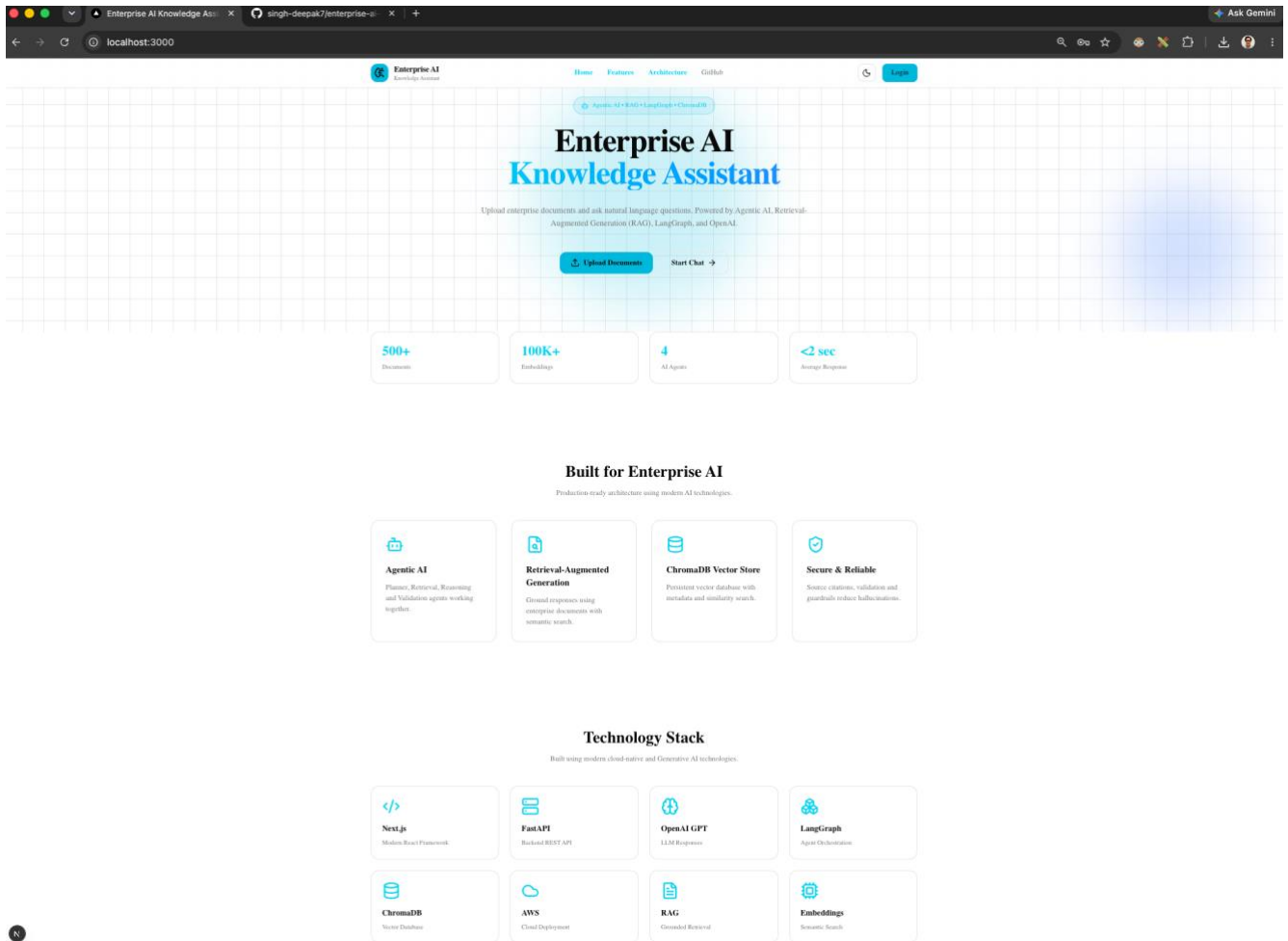
- filesystem document storage
- ChromaDB persistence
- application-level conversation memory
- local document metadata storage

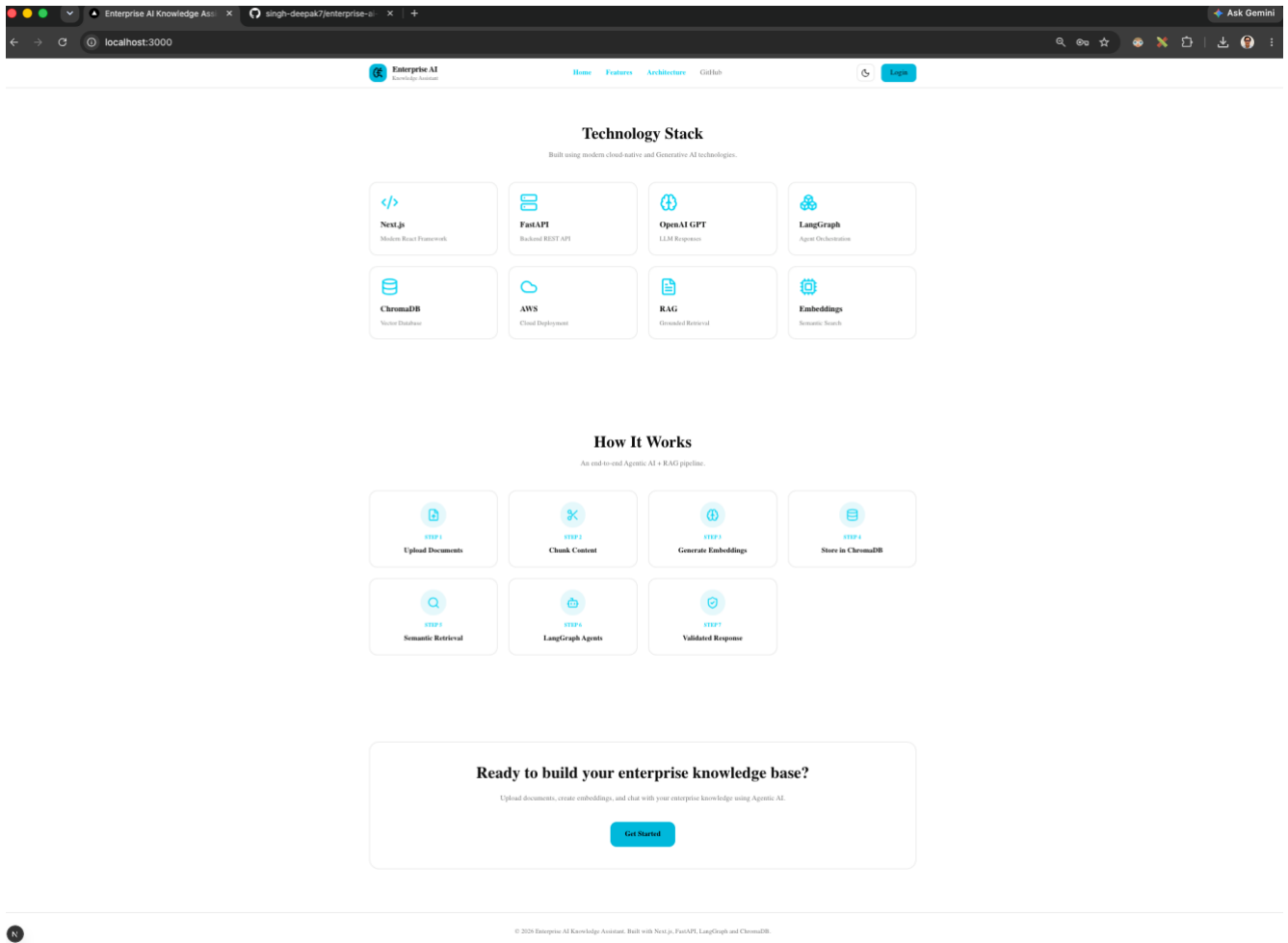
For a larger production deployment, these boundaries can be replaced with managed services without changing the overall application flow.

Examples include object storage for uploaded files, a managed vector database, persistent conversation storage, centralized application telemetry, authentication, authorization, and cloud deployment.

Demo

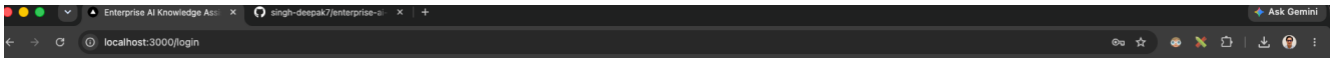
Welcome Page:





When you click on Upload Documents / Start Chat/ Login button user will be taken to sign in page

If



[← Back to home](#)

Sign in

Access the Enterprise AI Knowledge Assistant.

Username

Password

Sign in

If wrong credentials are passed

[← Back to home](#)

Sign in

Access the Enterprise AI Knowledge Assistant.

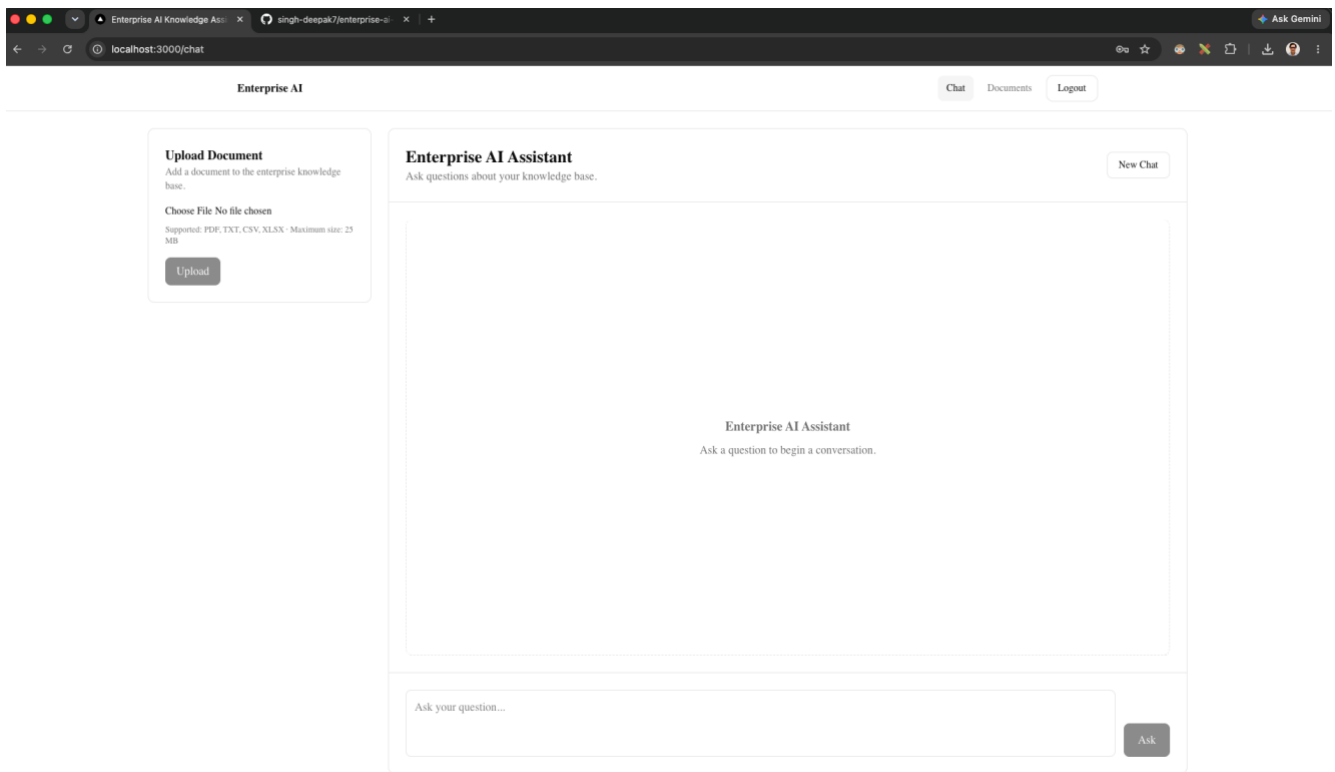
Username

Password

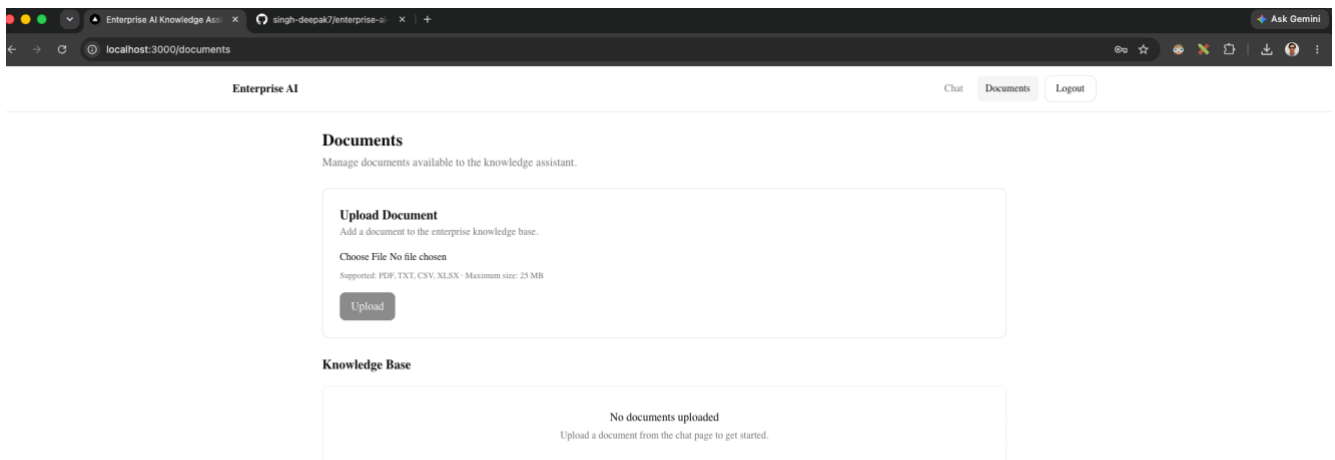
Invalid username or password.

Sign in

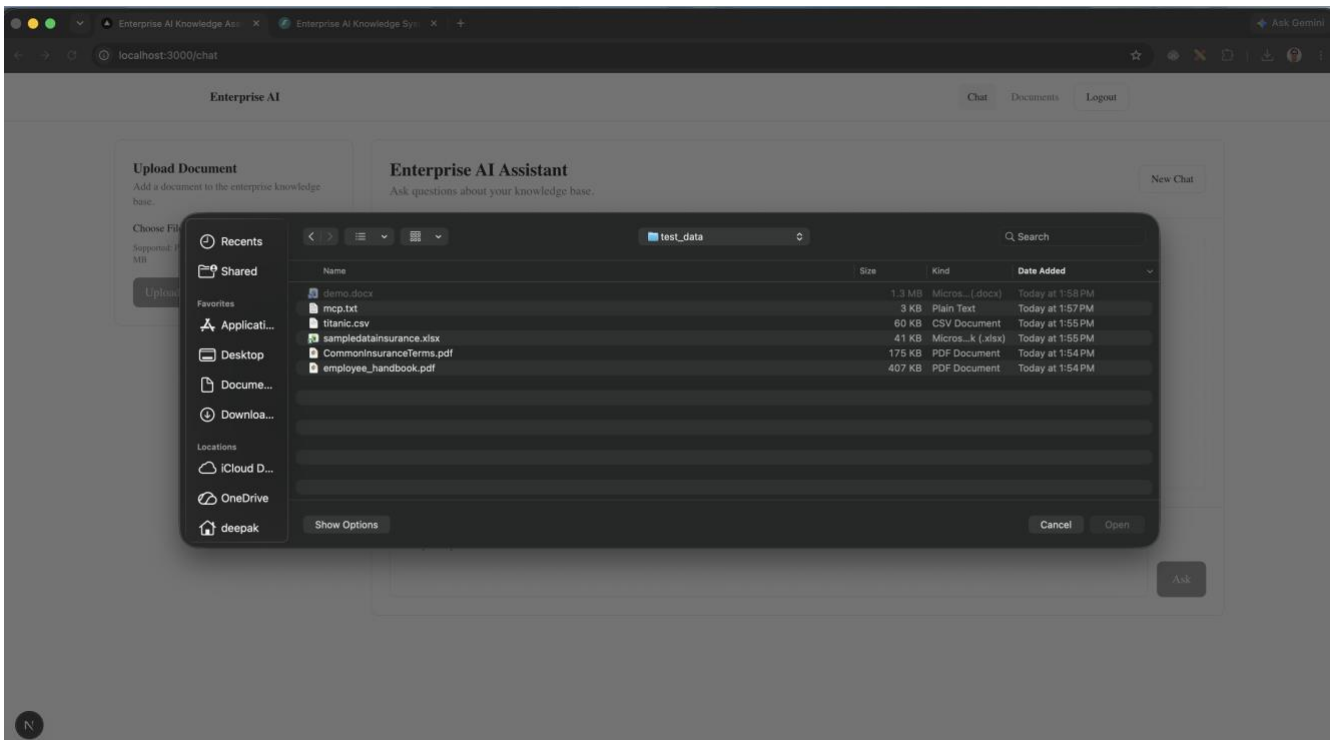
After successful login



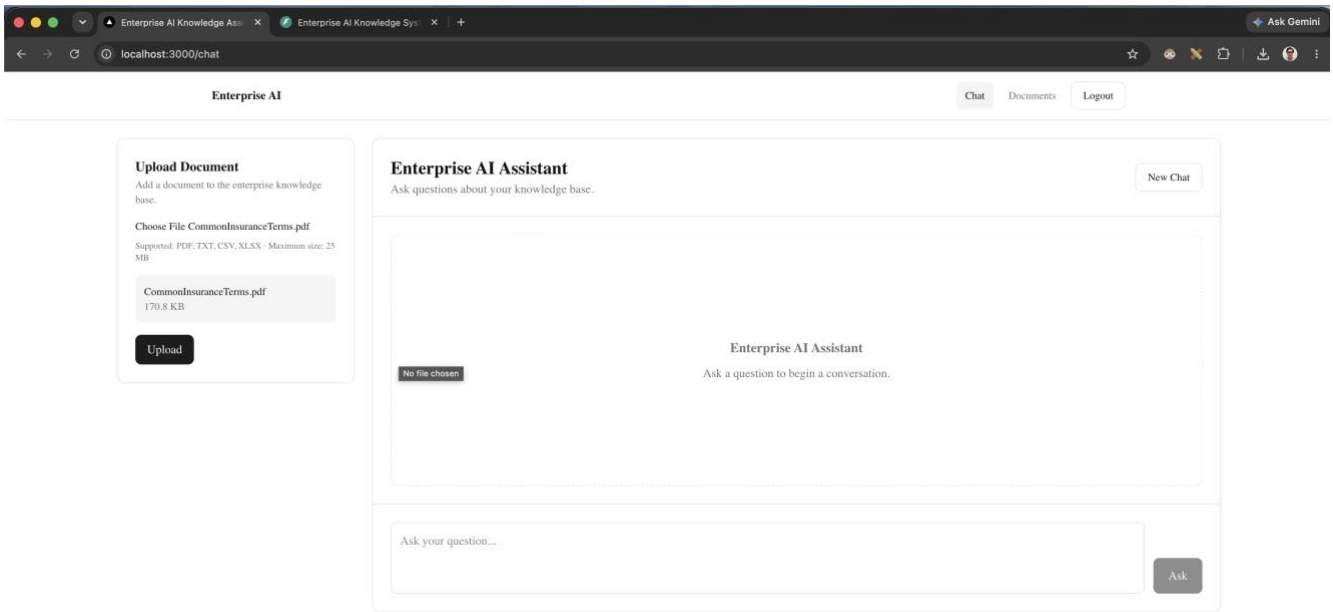
If you click on Documents link above, you see current documents knowledge base available.



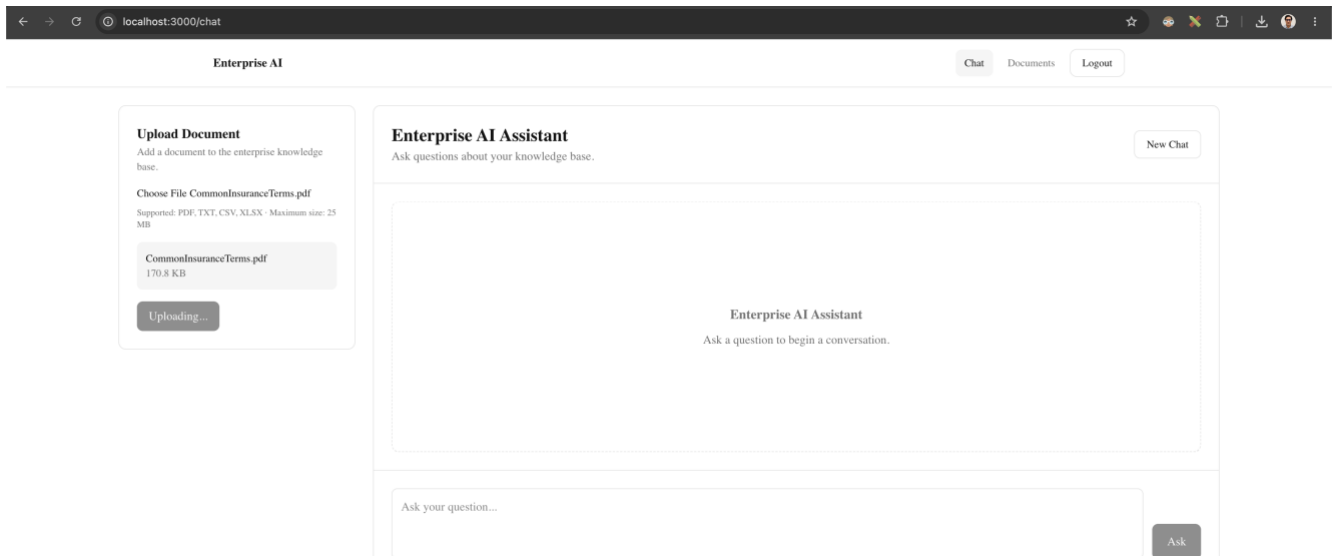
When we click choose a file and browse through files only .csv, .txt, .pdf and .xlsx files will be allowed to upload.



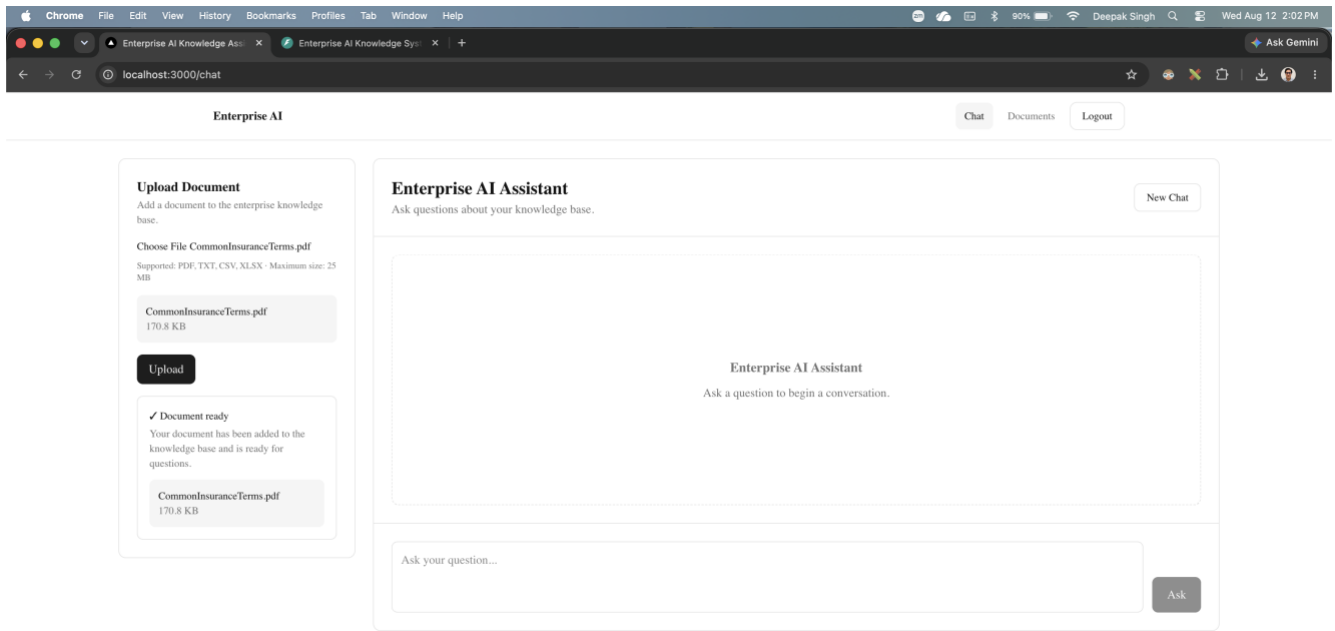
Lets Select CommonInsuranceTerm.pdf and test



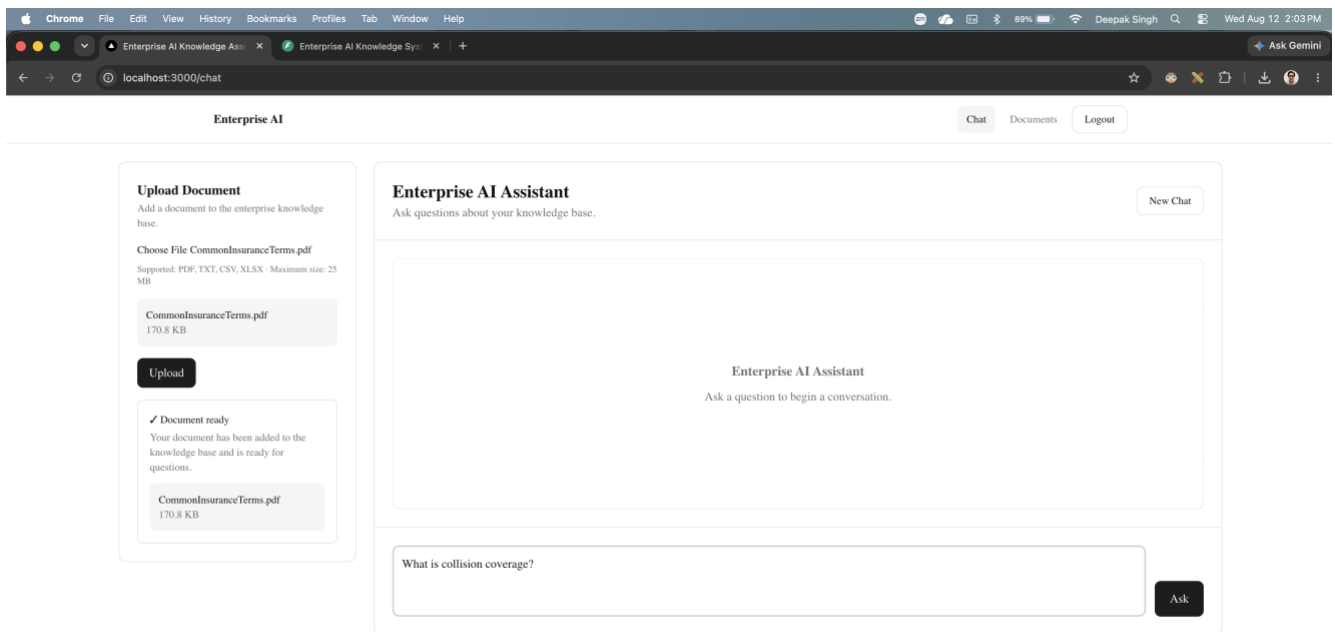
File gets uploaded.



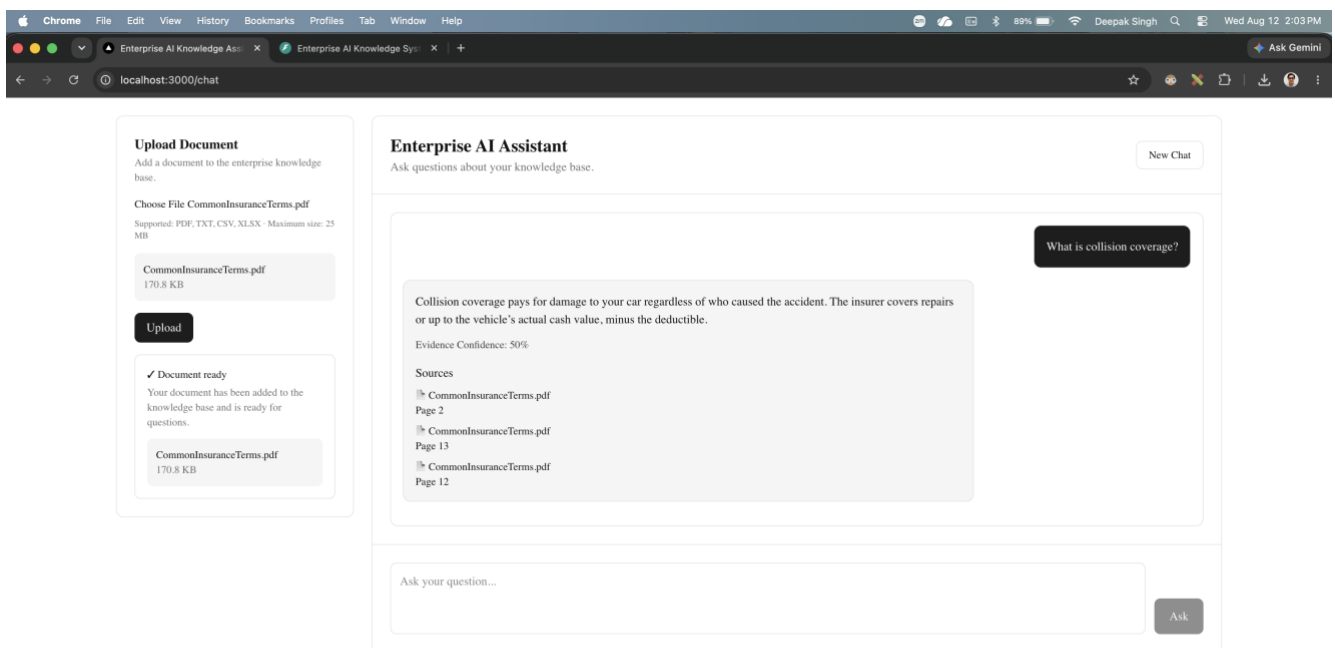
Once file is uploaded successfully Document ready message will be displayed. Now we can ask questions related to the document.



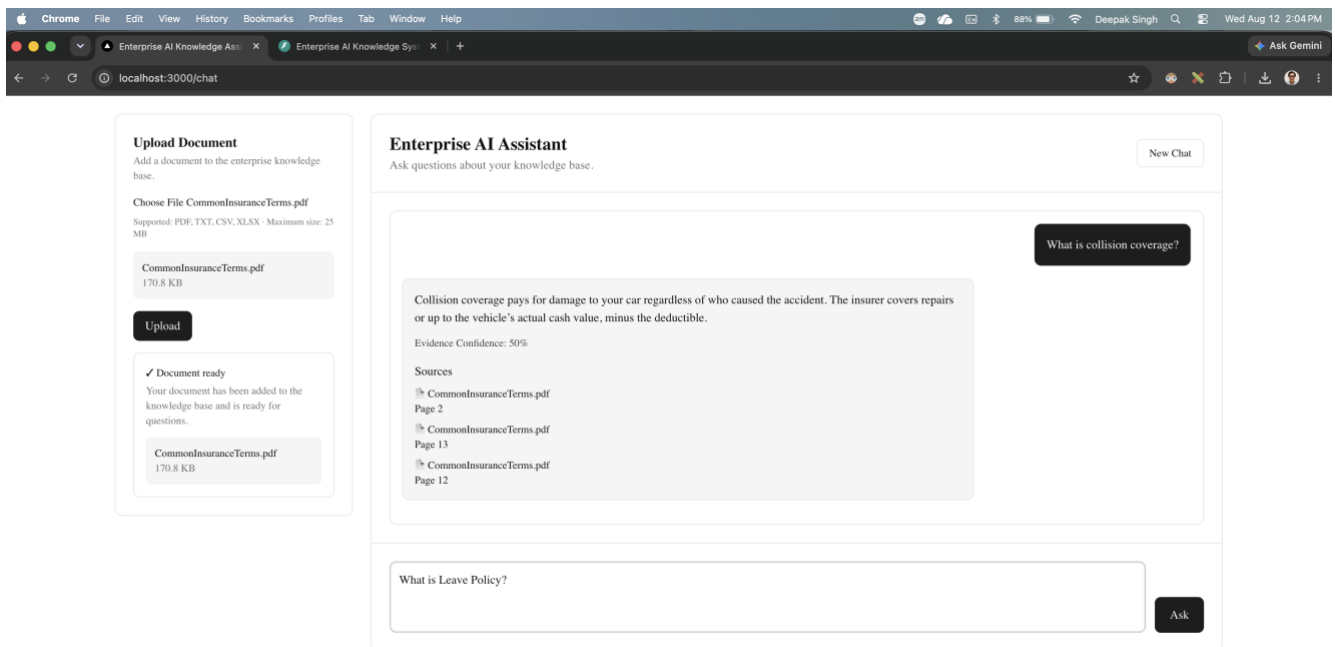
Let's try by asking What is collision coverage?



We will see the response along with citations.

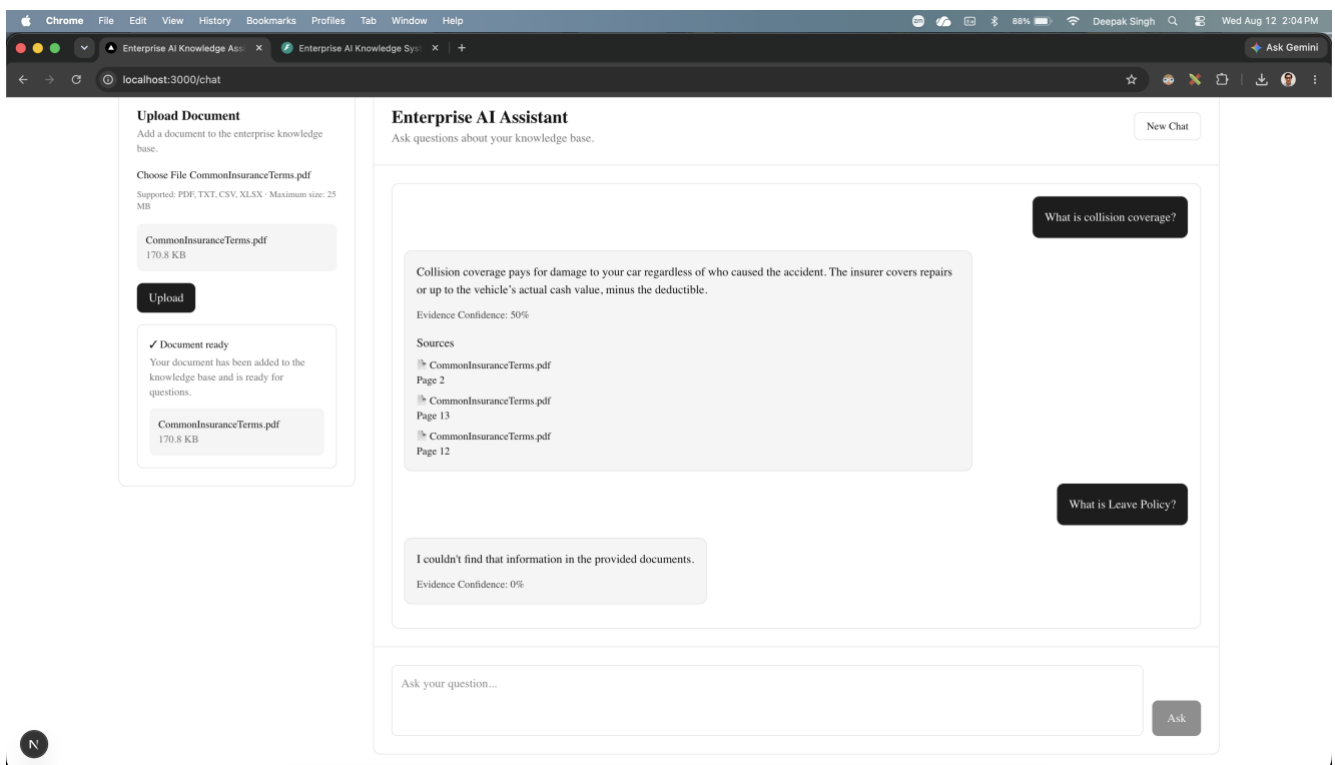


Now let's try to ask question not related to the document uploaded. Like What is leave policy?



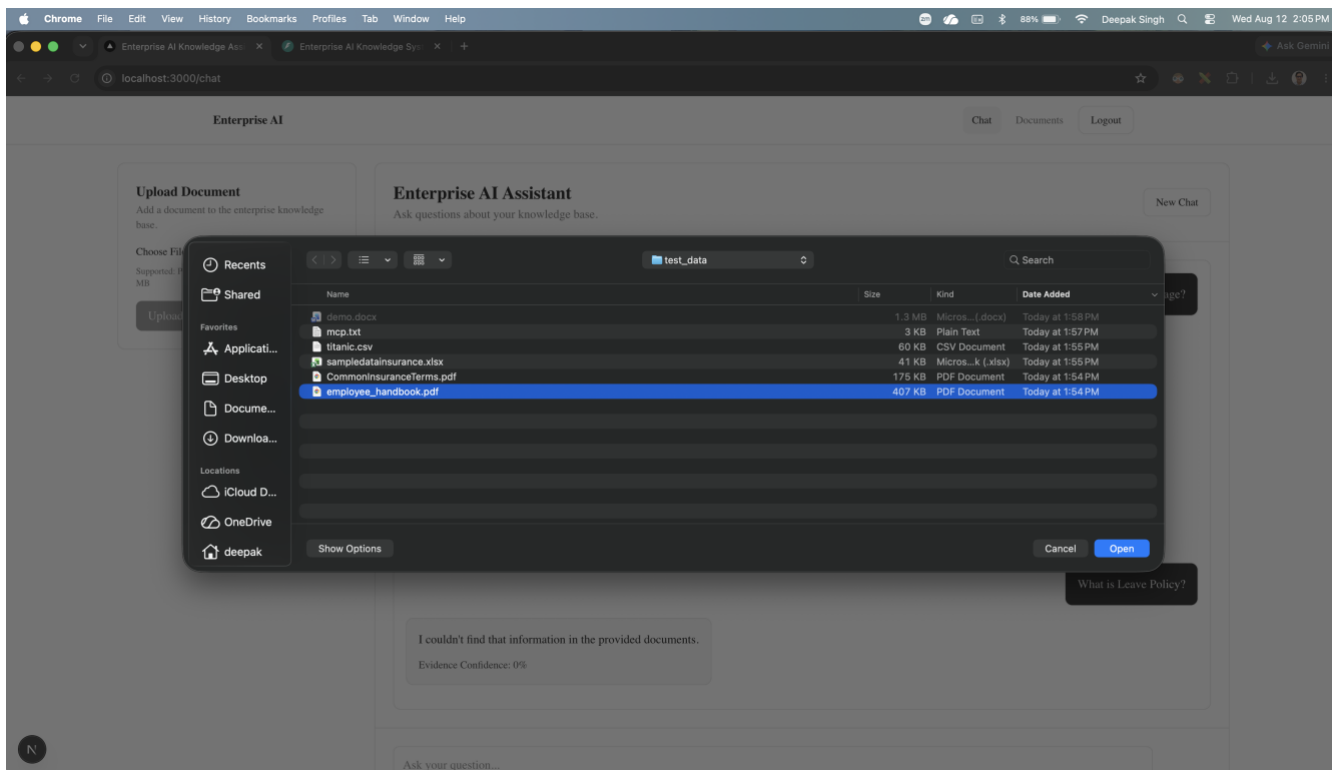
N

We will see response – I couldn't find the information in the provided document.

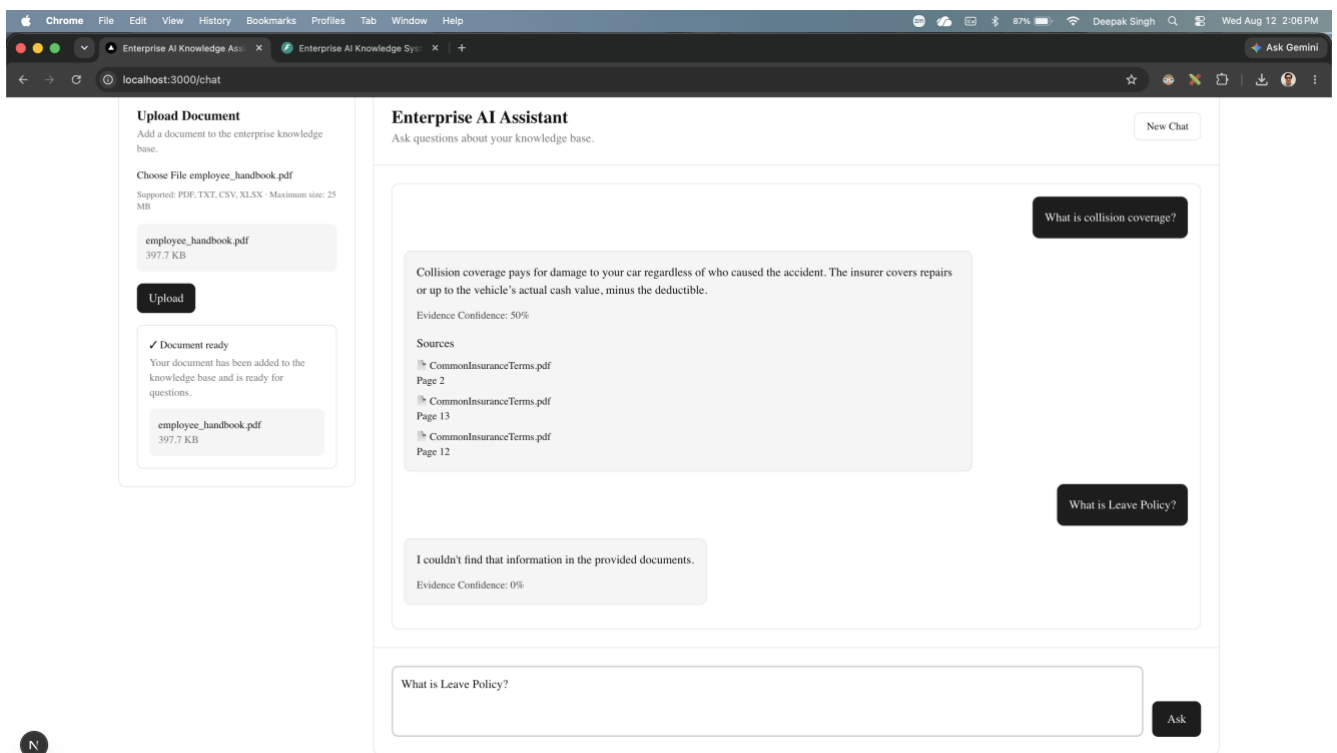


N

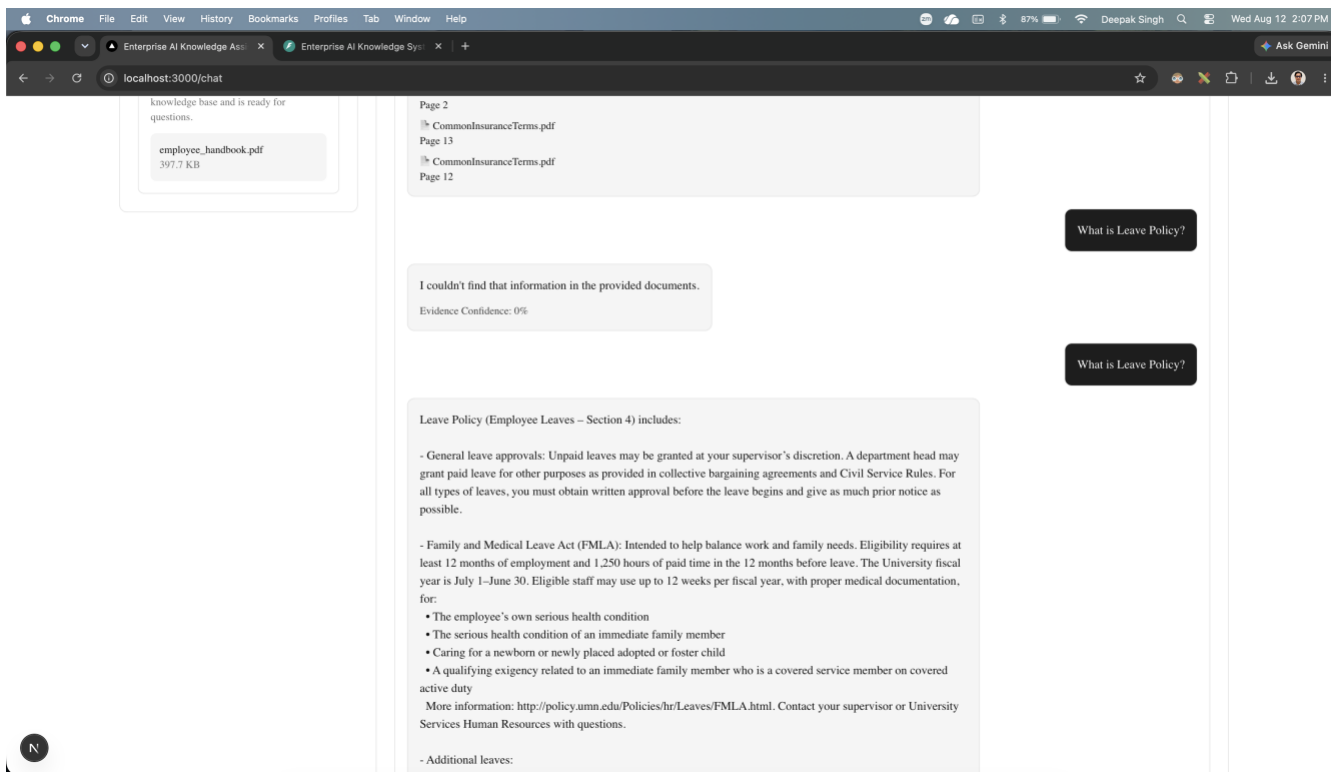
Now will upload `employee_handbook.pdf` which has the answer to previous question.



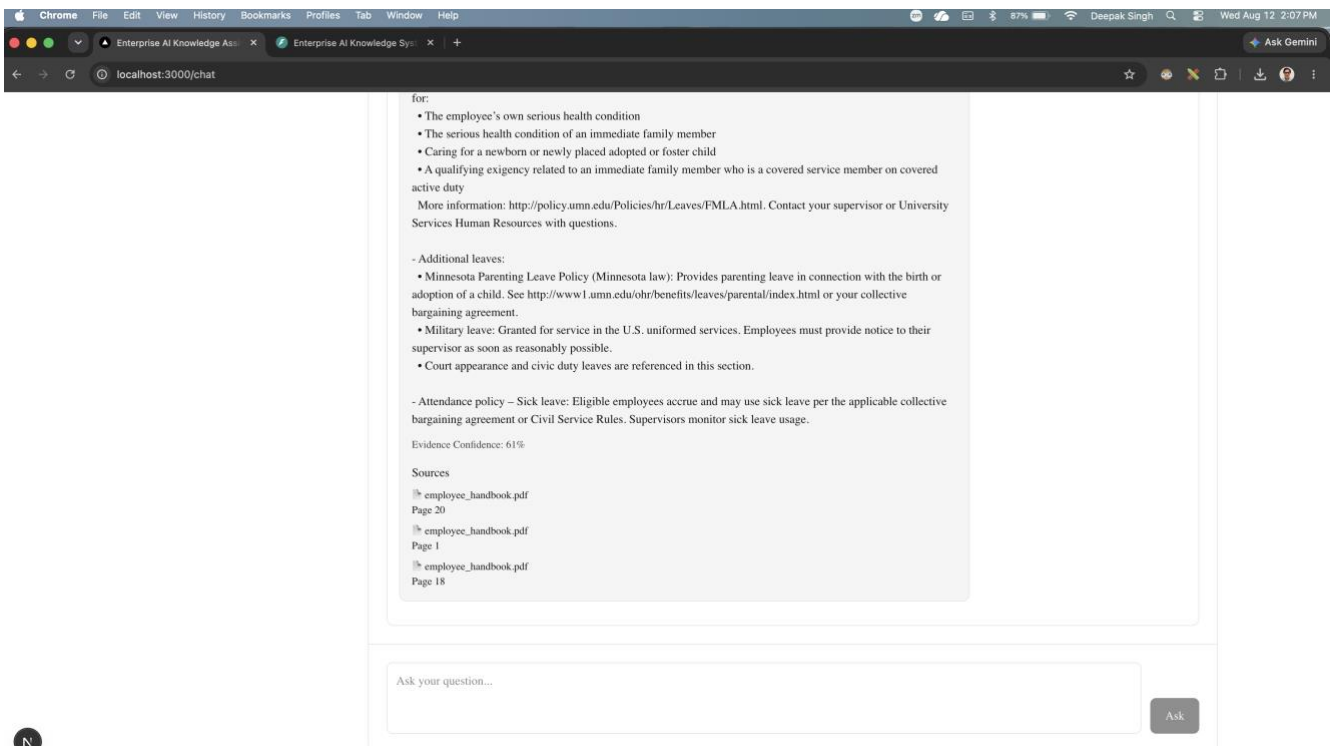
Once pdf gets uploaded successfully, will ask same question again.



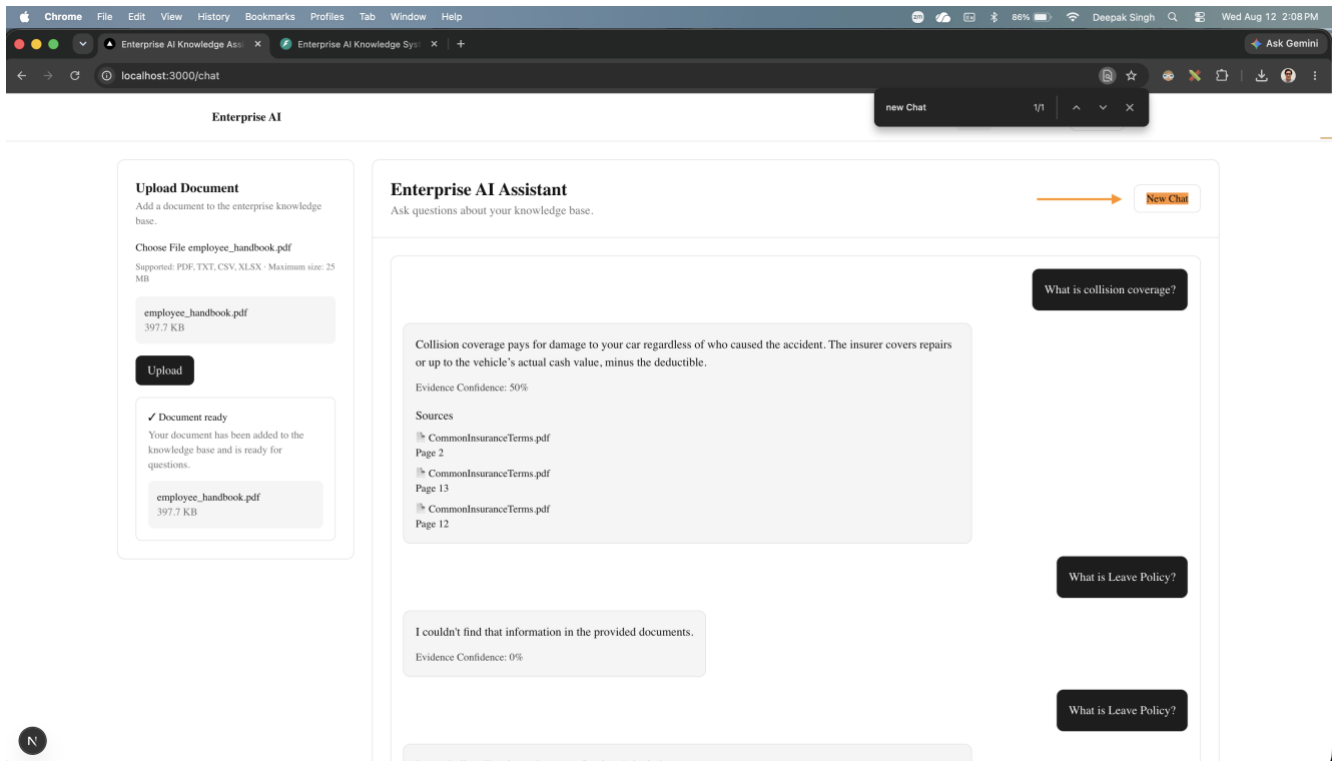
We should see answer now.



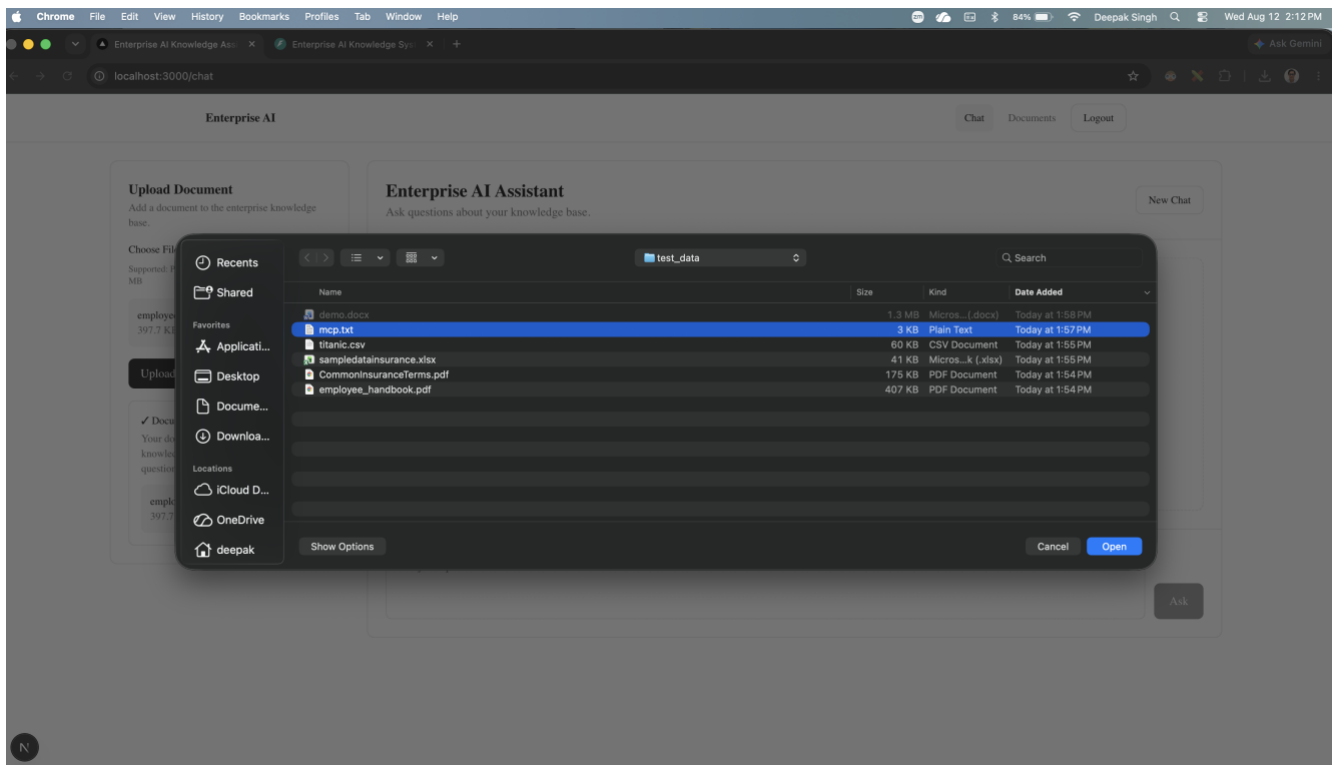
along with citations.



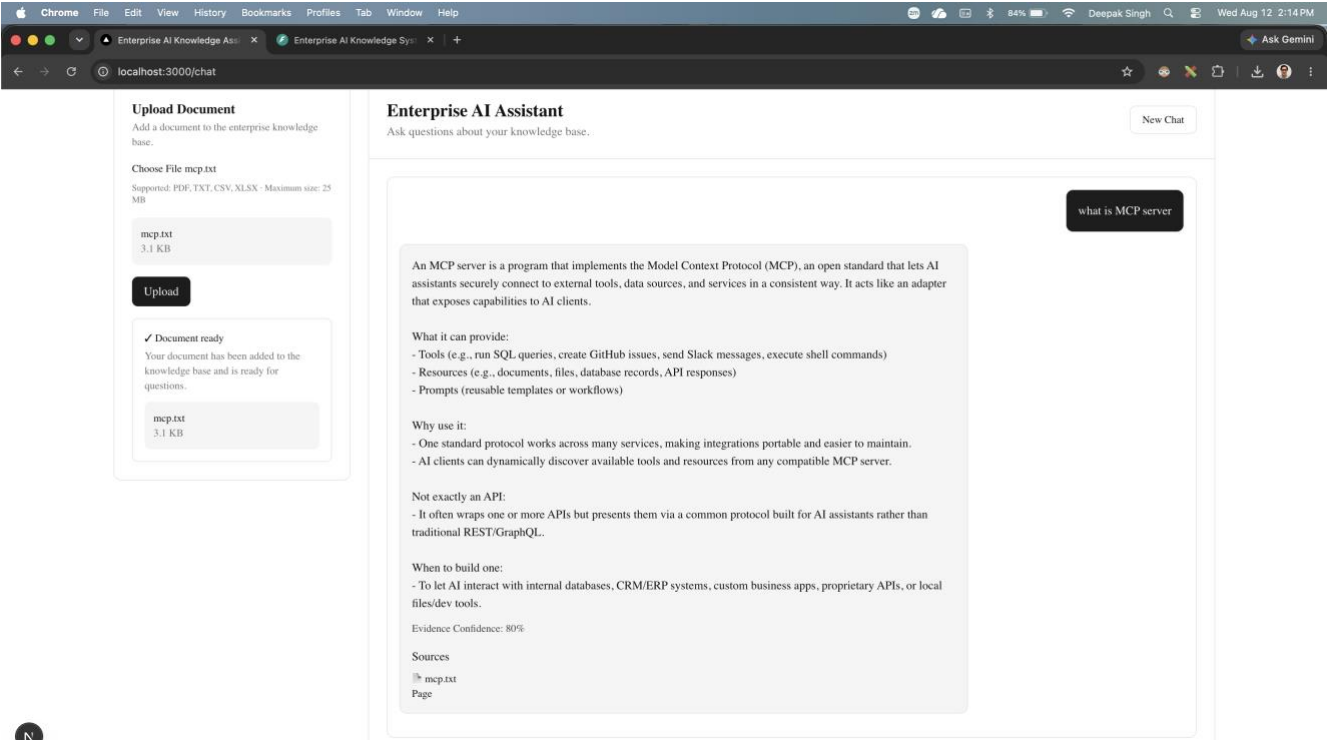
On the top there is a new chat option which will clear the old chat and start new session.



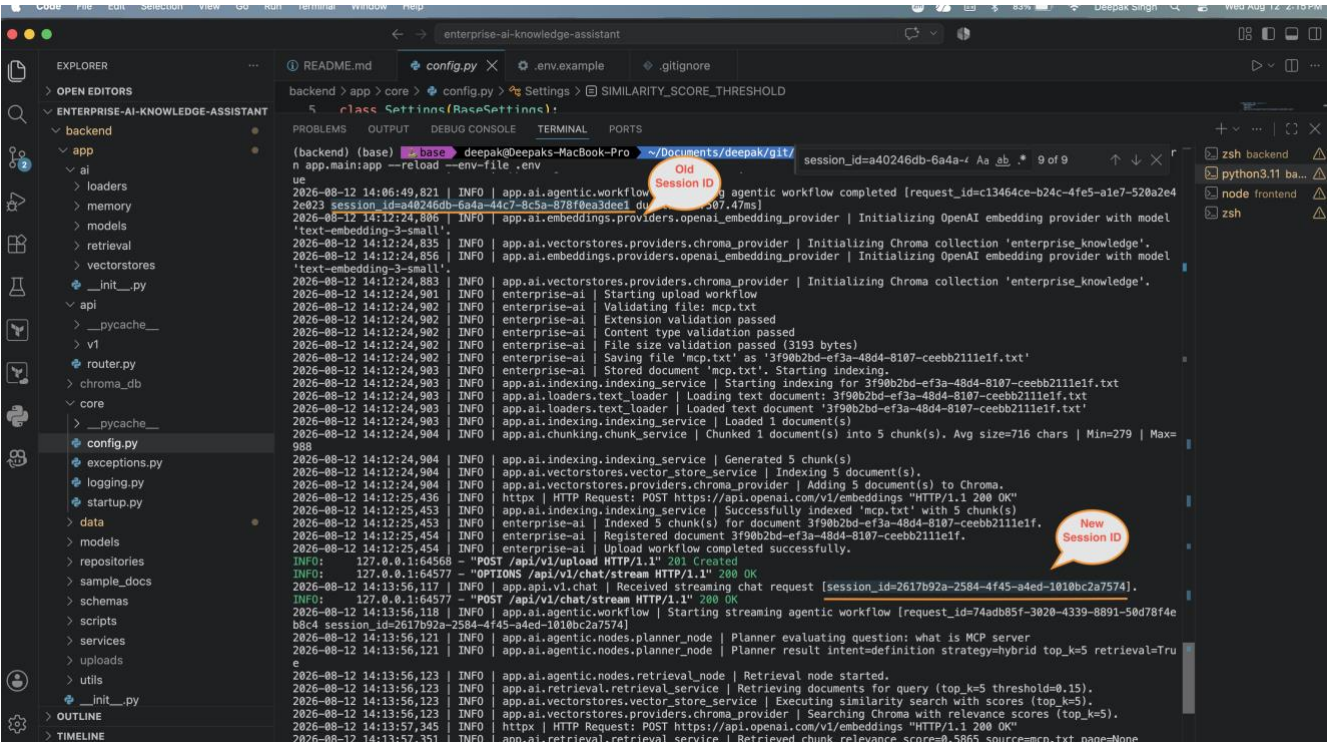
Lets upload mcp.txt and ask a new question in new session.



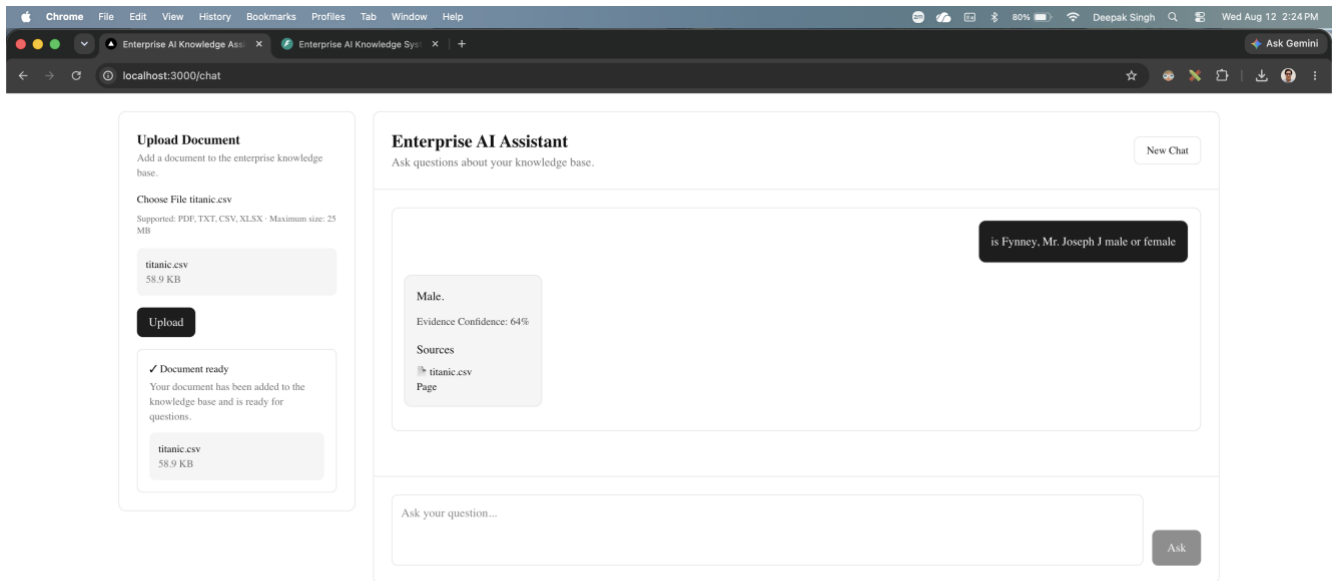
Will see the answer coming back



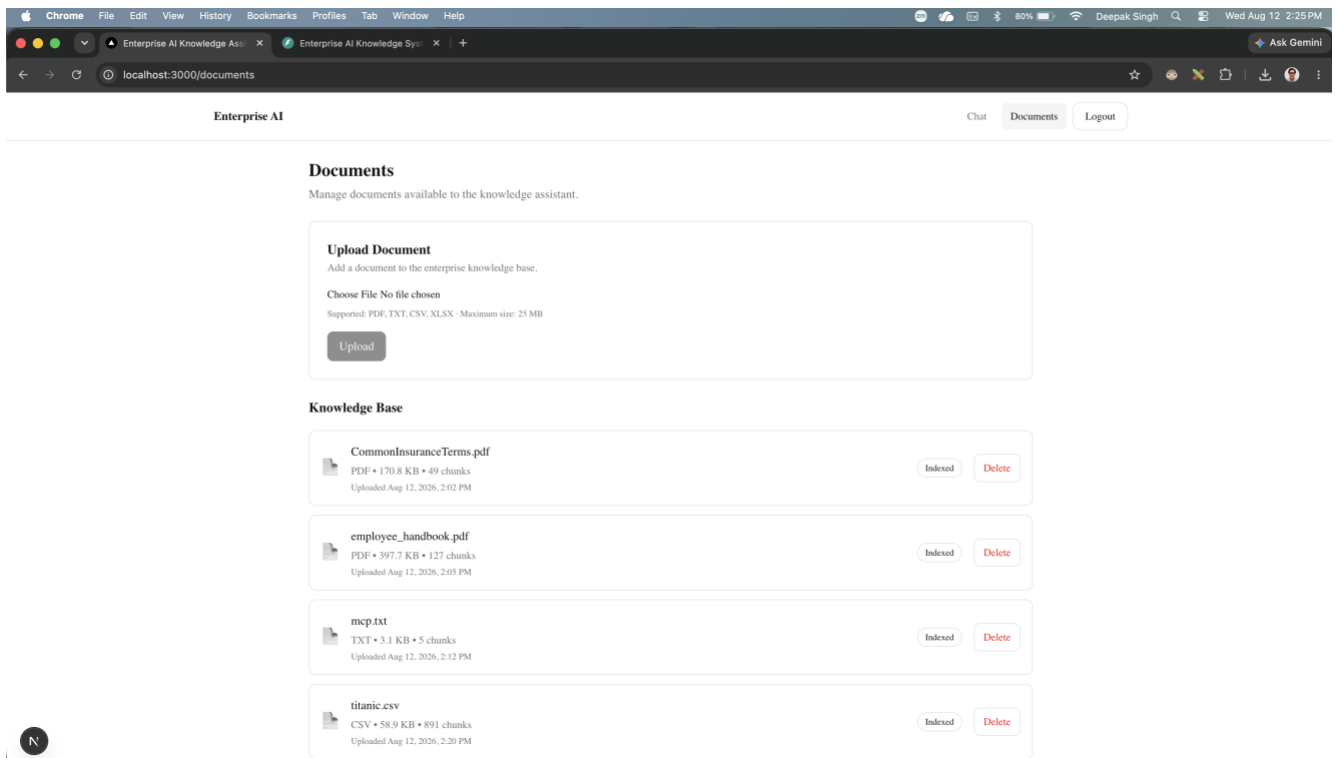
Banck end logs shows new session id being generated.



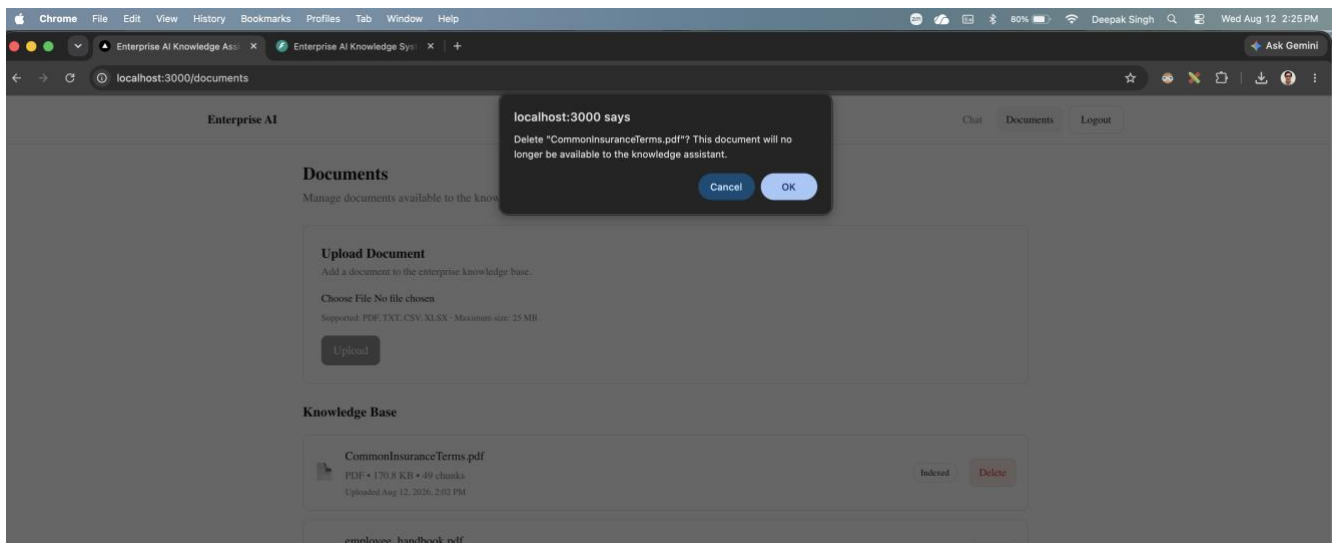
Now let's try by uploading titanic.csv file and ask question related to it and see answer.



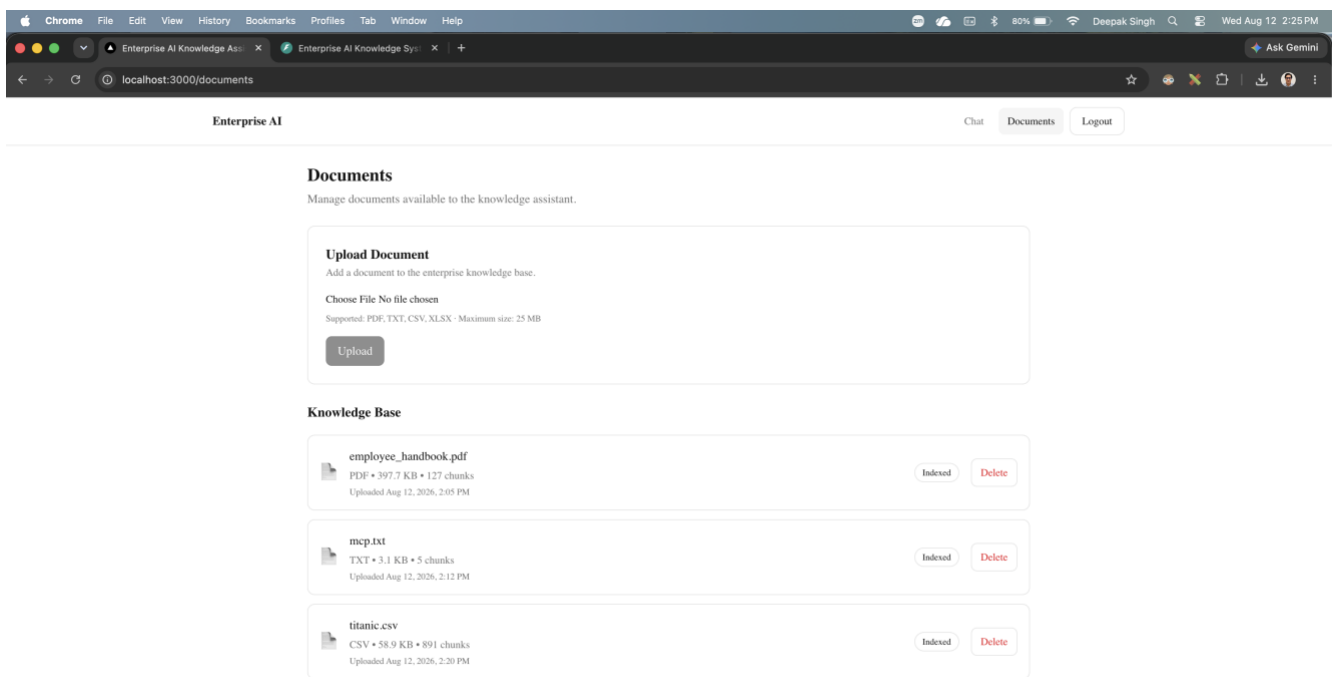
When we go to documents link we can see all the documents that were uploaded.



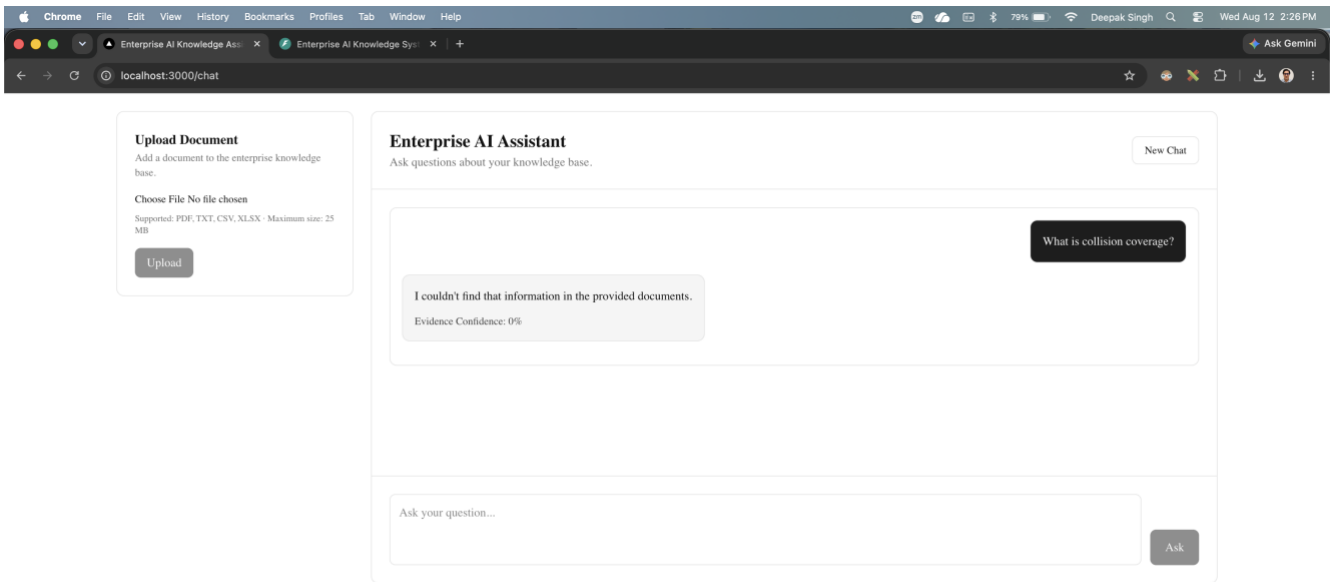
Let's try deleting CommonInsuranceTerms.pdf file.



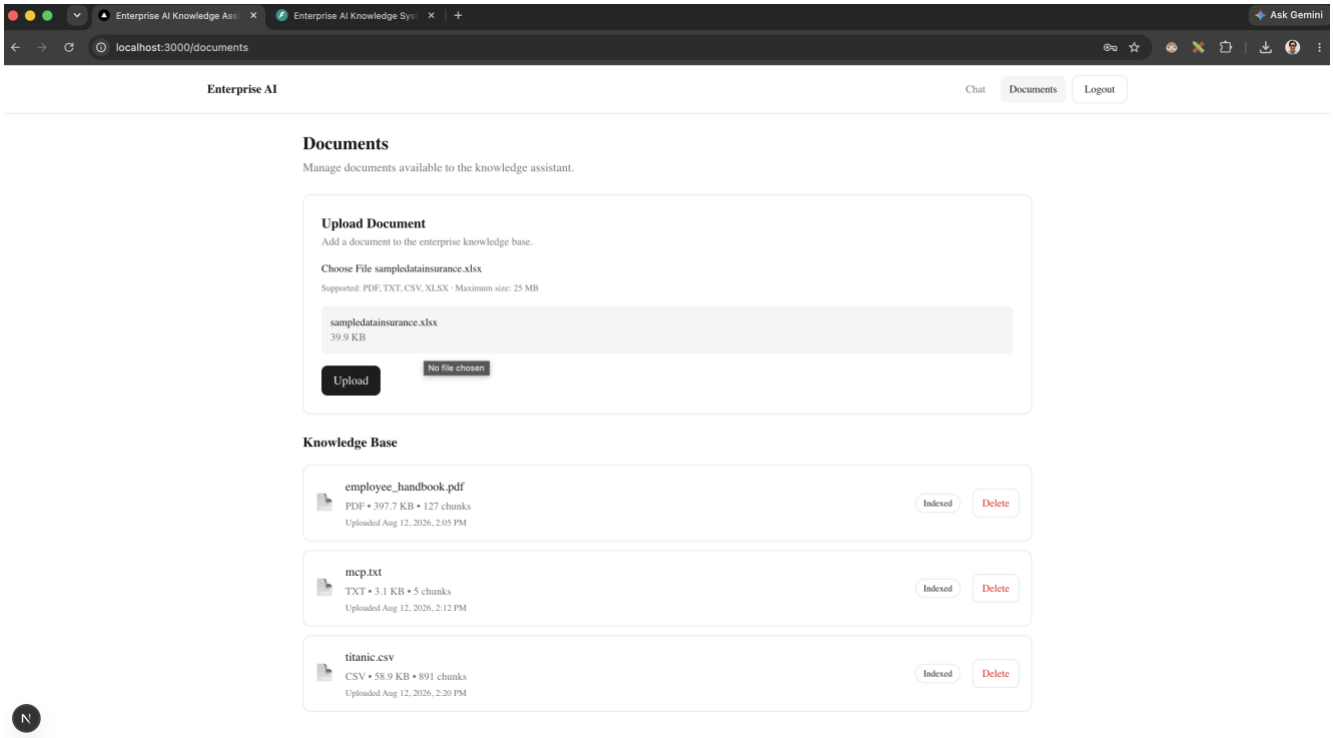
We can see pdf is no longer present.



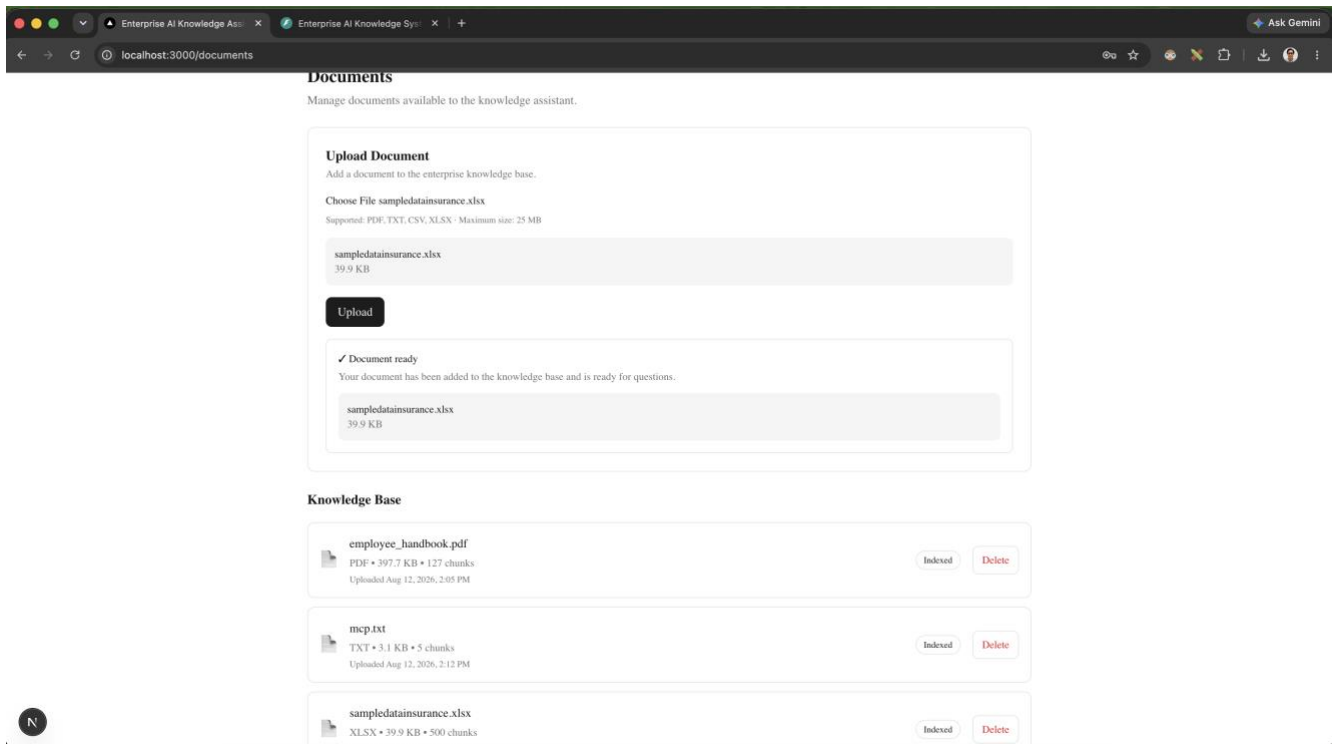
Let's go back to chat and ask same question again for which we got answer - What is collision coverage? We can see since document has been removed it's not able find the information.



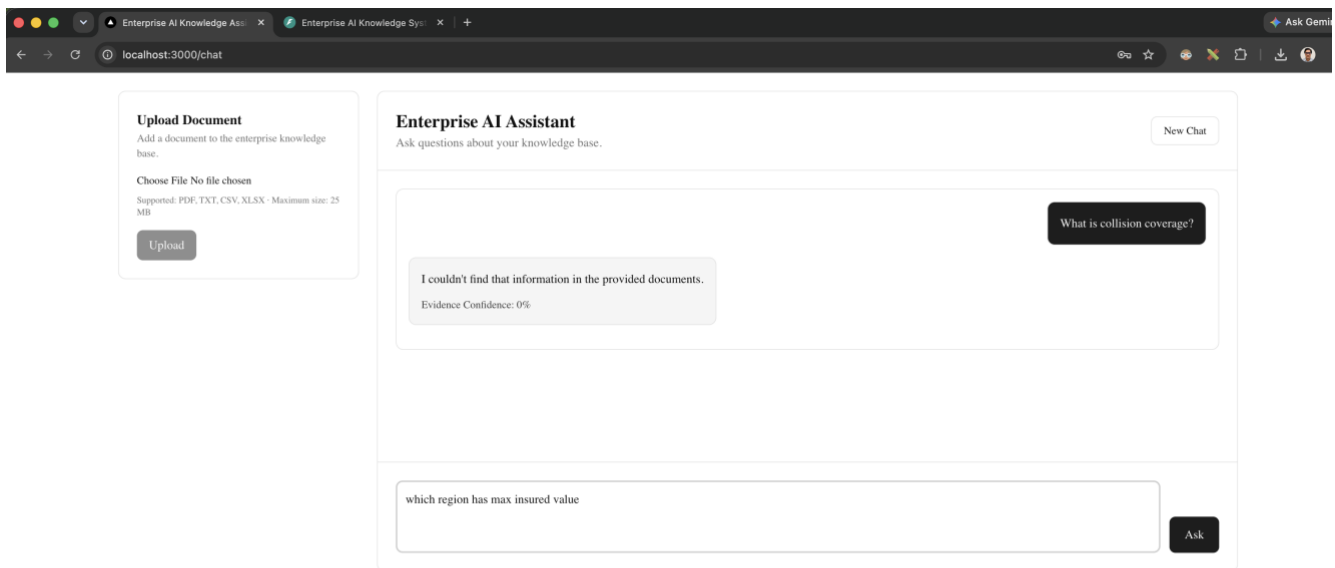
We can upload documents from Documents page too, let's upload sampledatainsurance.xlsx



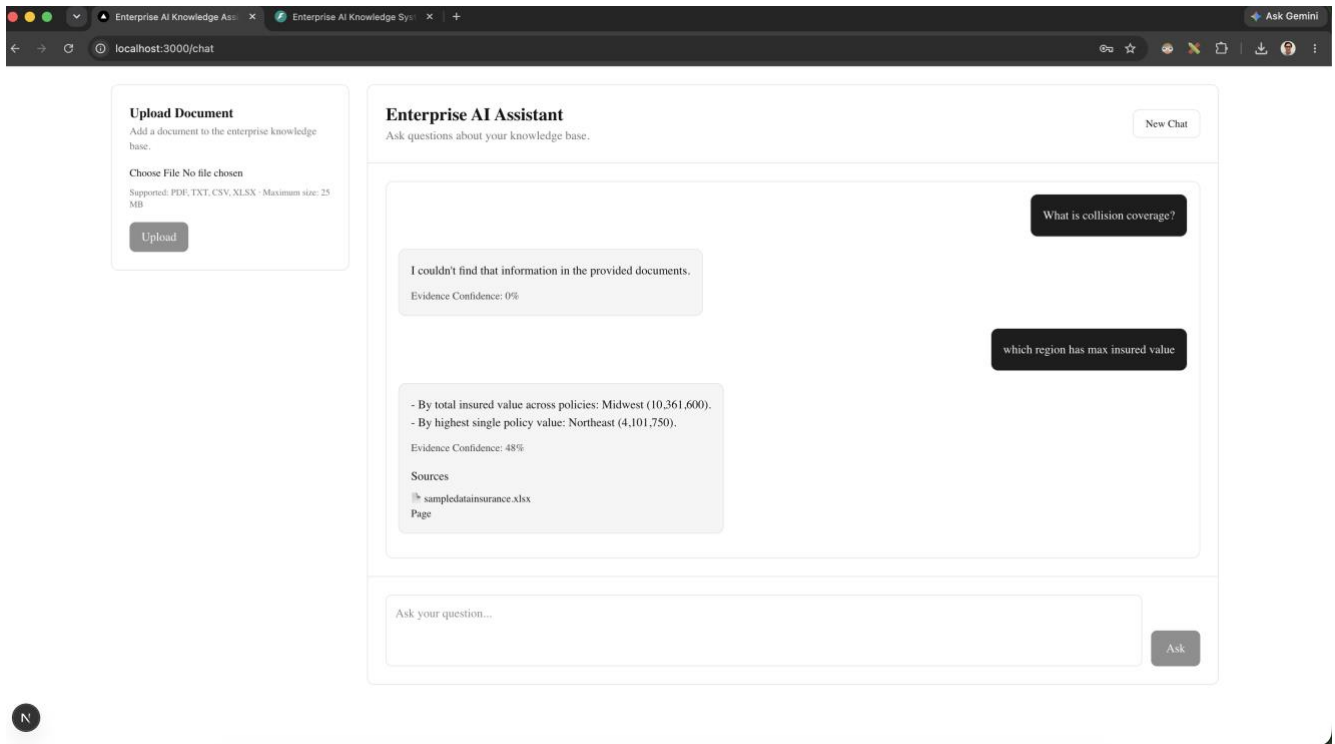
Once successfully uploaded we can see the document right away.



Now ask question related to that document. which region has max insured value?



Ans see answer right away



Future Enhancements

The current version focuses on demonstrating the complete Enterprise AI and RAG workflow locally. The architecture is designed so that local components can later be replaced with managed cloud services.

AWS Deployment

Deploy the application to AWS to provide a scalable and production-ready runtime environment.

The frontend and backend can be containerized and deployed using AWS services such as ECS or EKS. Production deployment can also include load balancing, health checks, centralized logging, monitoring, and automated CI/CD pipelines.

Persistent Application Storage

The current implementation uses local storage for application metadata and some runtime state.

A future version can replace local persistence with Amazon DynamoDB or another managed database service. This can store information such as:

- document metadata
- document processing status
- conversation sessions
- conversation history
- user information
- application configuration and audit information

Using persistent storage will allow application data to survive container restarts and support multiple backend instances.

Uploaded documents can also be moved from the local filesystem to Amazon S3.

Authentication and Role-Based Access

Add authentication so users must sign in before accessing the application.

After authentication, the application can determine the user's assigned role and permissions. Authorization rules can then control which functionality and documents each user can access.

For example, roles could include:

```
Administrator
|
+-- Manage users
+-- Upload and delete documents
+-- Access all documents
+-- Use the knowledge assistant

Standard User
|
+-- Access permitted documents
+-- Use the knowledge assistant
+-- View permitted conversation history
```

The backend should enforce these permissions rather than relying only on frontend controls.

Document retrieval can also become role-aware so the RAG pipeline only retrieves information the authenticated user is authorized to access.

AWS Secrets Manager

API keys and other sensitive configuration should not be stored directly in source code, committed environment files, or container images.

AWS Secrets Manager can be used to securely manage credentials such as:

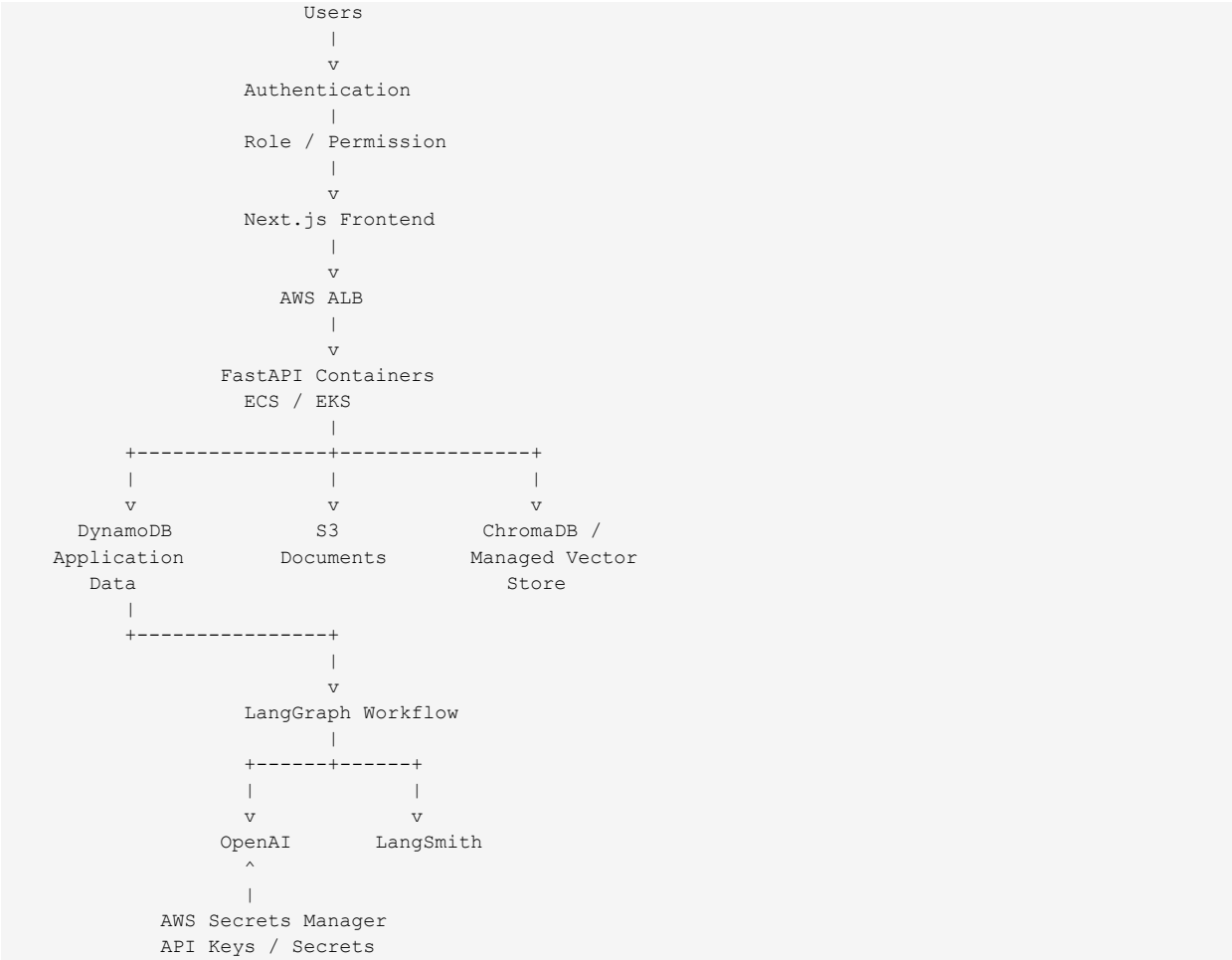
- OpenAI API keys
- LangSmith API keys
- database credentials
- external service credentials
- application secrets

The application can retrieve the required secrets during startup using its assigned AWS IAM role.

This removes the need to distribute production API keys through local `.env` files.

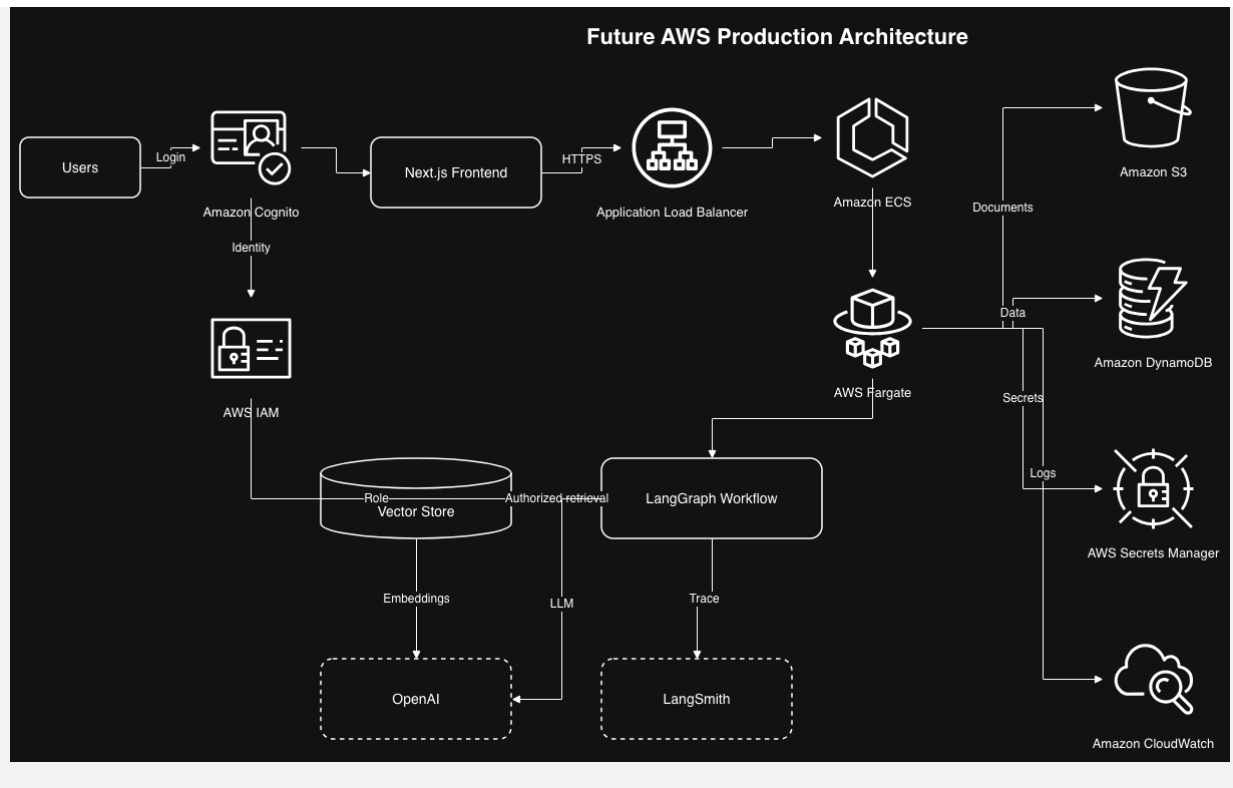
Production Target Architecture

A future AWS deployment could evolve toward the following architecture:



These enhancements would move the project from a locally running AI application toward a secure, persistent, scalable enterprise architecture.

AWS Architecture diagram



Code Repository

GitHub code repository – <https://github.com/singh-deepak7/enterprise-ai-knowledge-assistant>

GitHub portfolio - <https://singh-deepak7.github.io/enterprise-ai-knowledge-assistant>